

Tidy Ranges Tutorial

Michael Love

Invalid Date

1 Tidy Ranges Tutorial

About

I provide examples of computing operations on genomic ranges using the [tidy data philosophy](#).

These examples focus on the packages *dplyr*, *plyranges*, *nullranges*, *excluderanges*, as well as *tidy transcriptomics* packages such as *tidybulk* and *tidySummarizedExperiment*.

2 Bootstrap overlap

Objective: determine if one set of peaks are overlapping another set of peaks more or less than expected when comparing to sets of *null* features.

The null features can be generated in a variety of ways – here we will generate them by resampling large blocks of one of the peak sets. The motivation for sampling blocks, instead of placing features uniformly along the chromosome (“shuffling”), is to better preserve inter-feature distances, because genomic features tend to cluster in the genome, even after considering things like excluded regions. This technique of generating null feature sets is called *block bootstrap resampling*, and we will use the *nullranges* implementation of the block bootstrapping algorithm (Mu et al. 2023) to generate the null features, followed by overlap analysis with *plyranges* (Lee, Cook, and Lawrence 2019). The approach used in *nullranges* to generate bootstrap ranges closely follows the GSC method proposed by Bickel et al. (2010).

Note that an alternative approach for null hypothesis comparisons is to define a set of *covariate-matched ranges*, an approach also implemented in the *nullranges* package and described in Davis et al. (2023). For examples of using matching with ranges, see the related articles [here](#).

We start by loading the ENCODE kidney and bladder H3K27ac ChIP-seq peaks used in the previous analysis (Dunham et al. 2012).

```
library(AnnotationHub)
```

```
Loading required package: BiocGenerics
```

```
Loading required package: generics
```

```
Attaching package: 'generics'
```

```
The following objects are masked from 'package:base':
```

```
as.difftime, as.factor, as.ordered, intersect, is.element, setdiff,  
setequal, union
```

Attaching package: 'BiocGenerics'

The following objects are masked from 'package:stats':

IQR, mad, sd, var, xtabs

The following objects are masked from 'package:base':

anyDuplicated, aperm, append, as.data.frame, basename, cbind,
colnames, dirname, do.call, duplicated, eval, evalq, Filter, Find,
get, grep, grepl, is.unsorted, lapply, Map, mapply, match, mget,
order, paste, pmax, pmax.int, pmin, pmin.int, Position, rank,
rbind, Reduce, rownames, sapply, saveRDS, table, tapply, unique,
unsplit, which.max, which.min

Loading required package: BiocFileCache

Loading required package: dbplyr

```
ah <- AnnotationHub()  
kidney_pks <- ah[["AH43443"]]
```

loading from cache

```
require("rtracklayer")
```

```
bladder_pks <- ah[["AH44180"]]
```

loading from cache

```
save(kidney_pks, bladder_pks, file="data/peaks.rda")
```

```
load("data/peaks.rda")
```

We will additionally obtain an excluded region set, so that we avoid placing bootstrap features into regions of the genome that don't typically have features. A variety of possible exclude lists are provided by the *excluderanges* packages and available via *AnnotationHub*. Here we will use the `hg19.Crawford.wgEncodeDukeMapabilityRegionsExcludable` regions, as they are available for hg19, which is the genome used with the peak sets.

```
# get exclude list from ucsc tracks
exclude <- rtracklayer::import("https://hgdownload-test.gi.ucsc.edu/goldenPath/hg19/encodeDCC/exclude.bed")
save(exclude, file="data/exclude.rda")
```

To make the code more generic, we will rename the kidney peaks to `x` and the bladder peaks to `y`. We will be looking for overlaps with features in `x` as the query set: how many of the features in `x` overlap features in `y`?

The following code reduces our analysis to looking only at standard chromosomes, excluding the mitochondrial genome (too small for including in the block bootstrap).

```
library(GenomeInfoDb)
x <- kidney_pks
y <- bladder_pks
x <- keepStandardChromosomes(x)
seqlevels(x, pruning.mode="coarse") <- setdiff(seqlevels(x), "chrM")
seqlevels(y, pruning.mode="coarse") <- seqlevels(x)
seqlevels(exclude, pruning.mode="coarse") <- seqlevels(x)
```

We are mostly concerned with avoiding placing bootstrapped features in large regions in the exclude list, so we subset the exclude list to features larger than 500 bp. Why do we have `plyranges::` in front of `filter`? This is because there is a function in the *ensembldb* package that is also called *filter*, so it's a bit safer if we are using both packages to use the package name.

```
library(plyranges)
exclude <- exclude %>%
  filter(width(exclude) >= 500)
```

We also subset to the peaks for kidney and bladder which have q-value less than 0.001 and signal value greater than 9 (these are arbitrary filter values, just for demonstration).

For further analysis, we will need the features in `y` to be sorted, for the bootstrapping, here we sort both sets.

```
q_thr <- 3
s_thr <- 9
x <- x %>%
  filter(qValue > q_thr & signalValue > s_thr) %>%
  sort()
y <- y %>%
  filter(qValue > q_thr & signalValue > s_thr) %>%
  sort()
```

Now we can assess how many overlaps we observed between `x` and `y`:

```
obs <- x %>% mutate(n_overlaps = count_overlaps(., y))
obs %>% summarize(total = sum(n_overlaps))
```

```
DataFrame with 1 row and 1 column
      total
<integer>
1      5073
```

```
table( obs$n_overlaps )
```

	0	1	2	3	4	6
	9453	3966	486	39	3	1

We can check if any of the features of `y` fall in the excluded regions:

```
y %>% mutate(n_overlaps = count_overlaps(., exclude)) %>%
  summarize(total = sum(n_overlaps))
```

```
DataFrame with 1 row and 1 column
      total
<integer>
1         0
```

The following chunk of code does the bootstrapping of features in `y`. Here we subset first to metadata columns of interest (an `id` variable that we create, and the signal value which we rename to `signal`).

```
pks_to_boot <- y %>%
  mutate(id = seq_along(.)) %>%
  select(id, signal = signalValue)
```

The `bootRanges` function returns the bootstrap feature sets combined into one *GRanges* object – this tidy format facilitates downstream analysis as we will see. The bootstrap iteration is stored in the `iter` metadata column.

```
library(nullranges)
R <- 30 # number of iterations
set.seed(5) # set seed for reproducibility
boots <- bootRanges(pks_to_boot, blockLength=5e5, R=R, exclude=exclude)
boots
```

BootRanges object with 174764 ranges and 3 metadata columns:

	seqnames	ranges	strand		id	signal	iter
	<Rle>	<IRanges>	<Rle>		<integer>	<numeric>	<Rle>
[1]	chr1	2011397-2011809	*		931	14.42934	1
[2]	chr1	2011973-2012399	*		932	17.57052	1
[3]	chr1	2012505-2013073	*		933	9.64691	1
[4]	chr1	2503670-2505748	*		3752	20.84181	1
[5]	chr1	2505855-2507134	*		3753	15.29241	1
...	...	...	...	.	...	...	...
[174760]	chrY	57499850-57501632	*		5141	10.09210	30
[174761]	chrY	57937341-57937960	*		1062	15.98656	30
[174762]	chrY	58145916-58146540	*		264	15.23097	30
[174763]	chrY	58146662-58147585	*		265	12.26661	30
[174764]	chrY	58214091-58214746	*		266	9.69322	30

seqinfo: 24 sequences from hg19 genome

The default use above of the `exclude` argument is to drop bootstrapped ranges that overlap the exclude list.

We can examine properties of permuted `y` over iterations, and compare to the original `y`. To do so, we first add the original features as `iter=0`.

```
combined <- pks_to_boot %>%
  mutate(iter=0) %>%
  bind_ranges(boots) %>%
  select(iter)
```

Then compute summaries:

```
library(tibble)
stats <- combined %>%
  group_by(iter) %>%
  summarize(n = n(),
            sum_width=sum(width)/1e6) %>%
  as_tibble()
```

Original y vs bootstrap:

```
stats[1,]
```

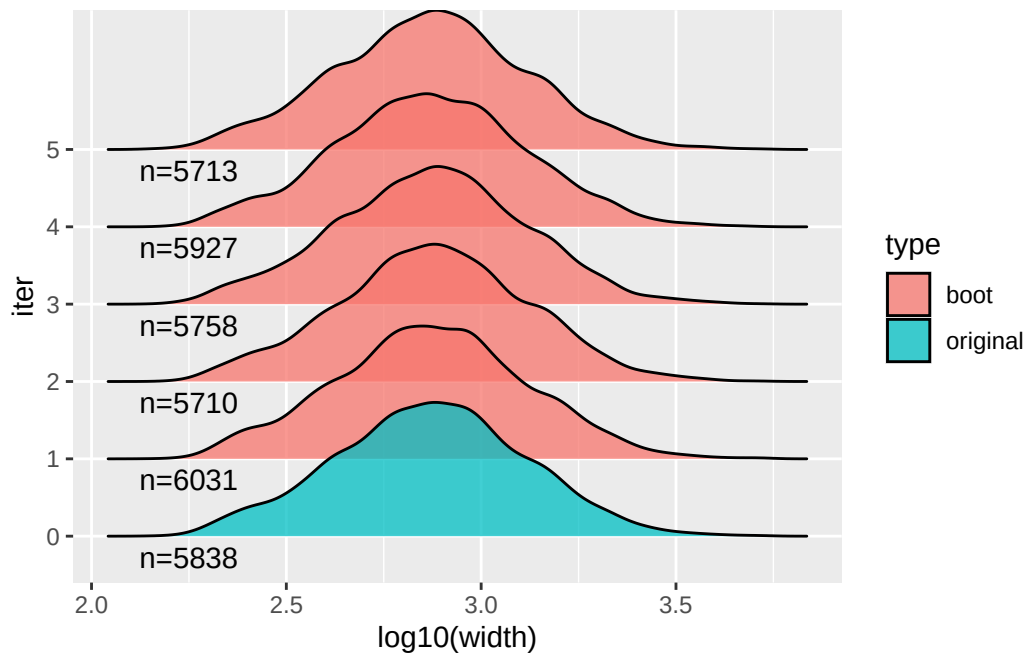
```
# A tibble: 1 x 3
  iter      n sum_width
<fct> <int>   <dbl>
1 0      5838     5.07
```

```
summary(stats[-1,])
```

	iter	n	sum_width
1	: 1	Min. :5595	Min. :4.843
2	: 1	1st Qu.:5723	1st Qu.:4.968
3	: 1	Median :5825	Median :5.042
4	: 1	Mean :5825	Mean :5.057
5	: 1	3rd Qu.:5910	3rd Qu.:5.130
6	: 1	Max. :6053	Max. :5.295
	(Other):24		

We can also look at distributions of various aspects, e.g. here the width of features, across a few of the bootstraps and the original feature set y.

```
library(ggplot2)
library(ggribes)
combined %>% filter(iter %in% 0:5) %>%
  select(iter) %>%
  as_tibble() %>%
  mutate(type = ifelse(iter == 0, "original", "boot")) %>%
  ggplot(aes(log10(width), iter, fill=type)) +
  geom_density_ridges(alpha = 0.75) +
  geom_text(data=head(stats),
            aes(x=2.25, y=iter, label=paste0("n=",n), fill=NULL),
            vjust=1.5)
```



To compute overlap with the null features, we need the `complete()` function from the *tidyr* package. We saw `complete()` before – this is used in the case that one of the iterations has no overlaps. In this case, we need to record the 0 value for proper inference and plots downstream. It is rare we would have no overlaps with so many features as we have in `x` and `y` but it's good practice to leave the `complete()` as part of the workflow so the code works correctly in all cases.

The overlap per iteration of the bootstrap is accomplished by a series of *plyranges* / *dplyr* commands (we switch to *dplyr* halfway through, after the `as_tibble()` call).

```
library(tidyr)
null <- x %>% join_overlap_inner(boots) %>%
  group_by(iter) %>%
  summarize(n_overlaps = n()) %>%
  as_tibble() %>%
  complete(iter, fill=list(n_overlaps = 0))
head(null)
```

```
# A tibble: 6 x 2
  iter n_overlaps
<fct>   <int>
1 1             60
2 2             60
3 3             76
```

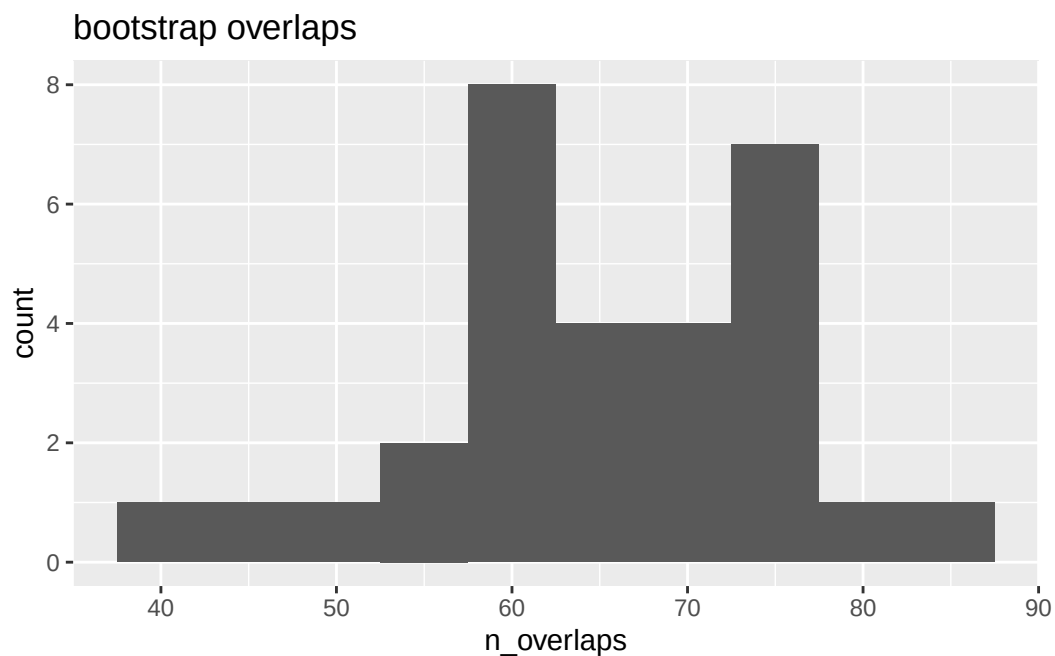
4	4	51
5	5	39
6	6	66

```
sum( obs$n_overlaps )
```

```
[1] 5073
```

The observed number of overlaps is about two orders of magnitude more than the bootstrapped number, which makes sense as two tissues would be expected to share a number of similar regulatory regions (as marked by H3K27ac) – more so than randomly placed genomic features, even after accounting for excluded regions and feature clustering.

```
ggplot(null, aes(n_overlaps)) +  
  geom_histogram(binwidth=5) +  
  ggtitle("bootstrap overlaps")
```



What could be improved with this analysis?

Note that in the above chunks where we count overlaps, we are doubly (or triply, etc.) counting features in **x** if they hit more than one feature in **y** or **boots**. We can count statistics per **x** feature by adding another **group_by** into the stream of operations. This also allows us to do

more complex operations, such as computing the maximum signal value for the overlapping features in `y` per feature in `x`:

First add an ID variable to keep track of `x` features:

```
x <- x %>% mutate(x_id = seq_along(x))
```

Then perform an inner join, and group by the new `x` ID:

```
obs <- x %>% join_overlap_inner(pks_to_boot) %>%  
  group_by(x_id) %>%  
  summarize(num_overlaps = n(),  
            max_signal = max(signal))  
sum( obs$num_overlaps > 0 )
```

```
[1] 4495
```

```
summary( obs$max_signal )
```

	Min.	1st Qu.	Median	Mean	3rd Qu.	Max.
	9.002	10.281	12.025	13.029	14.725	35.211

For the bootstrap ranges overlap step, we also need to add `iter` to the initial `group_by`, so we count per `x` feature and per iteration of the bootstrap:

```
null <- x %>% join_overlap_inner(boots) %>%  
  group_by(x_id, iter) %>%  
  summarize(num_overlaps = n()) %>%  
  as_tibble() %>%  
  group_by(iter) %>%  
  summarize(any_hits = sum(num_overlaps > 0)) %>%  
  complete(iter, fill=list(any_hits = 0))  
head(null)
```

```
# A tibble: 6 x 2  
  iter any_hits  
  <fct>   <int>  
1 1      55  
2 2      55  
3 3      68  
4 4      50  
5 5      37  
6 6      63
```

Still, we are seeing much more overlap in the observed data than in the bootstrap data:

```
sum( obs$num_overlaps > 0 ) /  
  mean(null$any_hits)
```

```
[1] 76.75014
```

The above code chunk then avoids double counting. We could also have made other per-x-feature statistics in the summarize step after the initial `group_by`, such as maximum signal of overlapping features.

What other ways could we have done this analysis? Suppose we don't just want the count of overlaps, but the rate of overlaps from the y perspective, keeping track of the variable number of features per bootstrap.

We demonstrate one approach to obtain this rate for the bootstraps:

```
x_thin <- x %>% select(x_id)  
null <- boots %>% select(id, iter) %>%  
  join_overlap_inner(x_thin) %>%  
  group_by(id, iter) %>%  
  summarize(num_overlaps = n()) %>%  
  as_tibble() %>%  
  group_by(iter) %>%  
  summarize(any_hits = sum(num_overlaps > 0)) %>%  
  complete(iter, fill=list(any_hits = 0))
```

Now combine with the per-iteration total count:

```
totals <- boots %>%  
  group_by(iter) %>%  
  summarize(total=n()) %>%  
  as_tibble()  
null %>% dplyr::left_join(totals) %>%  
  mutate(rate = any_hits/total)
```

Joining with ``by = join_by(iter)``

```
# A tibble: 30 x 4  
  iter any_hits total    rate  
  <fct>    <int> <int>   <dbl>
```

1	1	59	6031	0.00978
2	2	58	5710	0.0102
3	3	73	5758	0.0127
4	4	50	5927	0.00844
5	5	37	5713	0.00648
6	6	59	5888	0.0100
7	7	58	5910	0.00981
8	8	79	5836	0.0135
9	9	59	5770	0.0102
10	10	50	5991	0.00835

i 20 more rows

3 Compute overlaps

Objective: compute overlaps and summary statistics between two sets of genomic ranges. In particular, suppose we want to compute the mean genomic extent (distance from left-most to right-most basepair) of genes overlapping a set of query ranges.

We move on from the “classroom example” by seeing how we compute overlaps when the features are in genomic space. We will use *GRanges* in the Bioconductor package *GenomicRanges* (Lawrence et al. 2013) to represent the features and *plyranges* (Lee, Cook, and Lawrence 2019) to compute the overlaps, similarly to how we used *dplyr* to compute the overlaps in the previous analysis. So *data.frame* is to *dplyr* as *GRanges* is to *plyranges*.

```
library(plyranges)
```

Note the structure of the *GRanges* object. We can create a *GRanges* from a *data.frame* by specifying two of: `start`, `end`, or `width`.

```
df <- data.frame(
  seqnames="chr1",
  start=1 + c(34e6,36e6,36.6e6),
  width=c(2e5,2e5,1e5),
  strand=c("+","-","-"),
  range_id=factor(c("foo","bar","boo")))
r <- as_granges(df)
r
```

GRanges object with 3 ranges and 1 metadata column:

	seqnames	ranges	strand	range_id
	<Rle>	<IRanges>	<Rle>	<factor>
[1]	chr1	34000001-34200000	+	foo
[2]	chr1	36000001-36200000	-	bar
[3]	chr1	36600001-36700000	-	boo

seqinfo: 1 sequence from an unspecified genome; no seqlengths

In case you haven't seen this before, *GRanges* objects have specific functions to pull out information. See `?GRanges` for details.

```
length(r)
```

```
[1] 3
```

```
seqnames(r)
```

```
factor-Rle of length 3 with 1 run
  Lengths:    3
  Values  : chr1
Levels(1): chr1
```

```
strand(r)
```

```
factor-Rle of length 3 with 2 runs
  Lengths: 1 2
  Values  : + -
Levels(3): + - *
```

Let's use *plyranges* to find the genes that overlap a region of interest.

Typically, it is preferred to use Ensembl or GENCODE gene annotation, the latter of which can be obtained from *AnnotationHub*. Ensembl gene annotation can be easily manipulated with the *ensembladb* package ([Rainer, Gatto, and Weichenberger 2019](#)). Provided is some example code in an unevaluated code chunk:

```
library(ensembladb)
edb <- ... # obtain from AnnotationHub or from GTF file
g <- genes(edb)
```

Here, we will work with a static TxDb that is distributed as an annotation package in Bioconductor. We use this TxDb because it is an older genome release (hg19) that matches some ranges we will work with later, but generally it is recommended to use a recent (and versioned) Ensembl or GENCODE gene annotation.

```
library(TxDb.Hsapiens.UCSC.hg19.knownGene)
library(GenomeInfoDb)
```

The same function can be used to extract the gene ranges:

```
g <- genes(TxDb.Hsapiens.UCSC.hg19.knownGene)
```

24 genes were dropped because they have exons located on both strands of the same reference sequence or on more than one reference sequence, so cannot be represented by a single genomic range.

Use 'single.strand.genes.only=FALSE' to get all the genes in a GRangesList object, or use suppressMessages() to suppress this message.

```
g <- keepStandardChromosomes(g, pruning.mode="coarse")
g
```

GRanges object with 28622 ranges and 1 metadata column:

	seqnames	ranges	strand	gene_id
	<Rle>	<IRanges>	<Rle>	<character>
1	chr19	58856544-58874117	-	1
10	chr8	18248792-18258728	+	10
100	chr20	43213537-43280893	-	100
1000	chr18	25512843-25757910	-	1000
10000	chr1	243651535-244014381	-	10000
...	...	...	...	...
9991	chr9	114977351-115095933	-	9991
9992	chr21	35553030-35743680	+	9992
9993	chr22	19023795-19109967	-	9993
9994	chr6	90539613-90584155	+	9994
9997	chr22	50961997-50964890	-	9997

seqinfo: 25 sequences (1 circular) from hg19 genome

Now we are ready to test for overlaps. A left join gives us all the overlaps for ranges on the left side (here `r`). If a range on the left has no overlaps it appears with NA for the metadata columns of the right side ranges. If a range on the left side has multiple overlaps with the right side, it will appear multiple times in the output.

Keeping reading below on how to deal with this, if it is desired to have statistics on per-range overlaps.

```
r %>% join_overlap_left(g)
```

GRanges object with 11 ranges and 2 metadata columns:

seqnames	ranges	strand	range_id	gene_id
----------	--------	--------	----------	---------

	<Rle>	<IRanges>	<Rle>		<factor>	<character>
[1]	chr1	34000001-34200000	+		foo	114784
[2]	chr1	36000001-36200000	-		bar	105378645
[3]	chr1	36000001-36200000	-		bar	127703
[4]	chr1	36000001-36200000	-		bar	23154
[5]	chr1	36000001-36200000	-		bar	339488
[6]	chr1	36000001-36200000	-		bar	5690
[7]	chr1	36000001-36200000	-		bar	63967
[8]	chr1	36000001-36200000	-		bar	79932
[9]	chr1	36600001-36700000	-		boo	27095
[10]	chr1	36600001-36700000	-		boo	55700
[11]	chr1	36600001-36700000	-		boo	9967

seqinfo: 1 sequence from an unspecified genome; no seqlengths

If we want to exclude the zero matches cases, we can use an inner join:

```
r %>% join_overlap_inner(g)
```

GRanges object with 11 ranges and 2 metadata columns:

	seqnames	ranges	strand		range_id	gene_id
	<Rle>	<IRanges>	<Rle>		<factor>	<character>
[1]	chr1	34000001-34200000	+		foo	114784
[2]	chr1	36000001-36200000	-		bar	105378645
[3]	chr1	36000001-36200000	-		bar	127703
[4]	chr1	36000001-36200000	-		bar	23154
[5]	chr1	36000001-36200000	-		bar	339488
[6]	chr1	36000001-36200000	-		bar	5690
[7]	chr1	36000001-36200000	-		bar	63967
[8]	chr1	36000001-36200000	-		bar	79932
[9]	chr1	36600001-36700000	-		boo	27095
[10]	chr1	36600001-36700000	-		boo	55700
[11]	chr1	36600001-36700000	-		boo	9967

seqinfo: 1 sequence from an unspecified genome; no seqlengths

What about other types of overlap? We can specify a distance, called a “gap”, if we want to also find when ranges are *near* each other, up to a maximum allowed distance. We can also specify the minimum amount of overlapping basepairs. Every overlap function in plyranges has arguments `maxgap` and `minoverlap`. For example, to see if ranges are 50kb from each other, we would specify `maxgap=5e4`. If we want to know if ranges are 50kb from a particular endpoint of

another set of ranges, for example TSS, we could perform the operations `anchor_5p()` followed by `mutate(width=1)`, before overlapping the sets.

We can also perform summarization by columns either in the `r` or the `g` object:

```
r %>% join_overlap_inner(g) %>%
  group_by(range_id) %>%
  summarize(count=n())
```

DataFrame with 3 rows and 2 columns

	range_id	count
	<factor>	<integer>
1	bar	7
2	boo	3
3	foo	1

This is giving us the same information as the following:

```
r %>% count_overlaps(g)
```

```
[1] 1 7 3
```

Which can be added to the range data with a `mutate` call:

```
r %>% mutate(overlaps = count_overlaps(., g))
```

GRanges object with 3 ranges and 2 metadata columns:

	seqnames	ranges	strand	range_id	overlaps
	<Rle>	<IRanges>	<Rle>	<factor>	<integer>
[1]	chr1	34000001-34200000	+	foo	1
[2]	chr1	36000001-36200000	-	bar	7
[3]	chr1	36600001-36700000	-	boo	3

seqinfo: 1 sequence from an unspecified genome; no seqlengths

Up to this point, we could have used R's native pipe `|>` in place of the *magrittr* pipe `%>%`. However the above line won't work with R's native pipe, even if we use the underscore placeholder `_` that goes with R's native pipe, instead of the dot placeholder `..`. This is because R's native pipe doesn't allow the placeholder to be within a nested expression (`count_overlaps` called before `mutate`). In addition, R's native pipe only allows use of the placeholder once; sometimes it is convenient to access the incoming object more than once.

If we don't care about multiple overlaps, but just want a binary variable that records if there was one or more overlaps or not, we can ask if the count of overlaps is greater than 0:

```
r %>% mutate(overlaps_any = count_overlaps(., g) > 0)
```

GRanges object with 3 ranges and 2 metadata columns:

	seqnames	ranges	strand	range_id	overlaps_any
	<Rle>	<IRanges>	<Rle>	<factor>	<logical>
[1]	chr1	34000001-34200000	+	foo	TRUE
[2]	chr1	36000001-36200000	-	bar	TRUE
[3]	chr1	36600001-36700000	-	boo	TRUE

seqinfo: 1 sequence from an unspecified genome; no seqlengths

If we want to keep the information about the gene ranges, we swap the order of the ranges in the command:

```
g %>% join_overlap_inner(r)
```

GRanges object with 11 ranges and 2 metadata columns:

	seqnames	ranges	strand	gene_id	range_id
	<Rle>	<IRanges>	<Rle>	<character>	<factor>
105378645	chr1	36035408-36043409	-	105378645	bar
114784	chr1	33979598-34631443	-	114784	foo
127703	chr1	36179478-36184495	-	127703	bar
23154	chr1	36023074-36032875	+	23154	bar
27095	chr1	36602171-36621654	-	27095	boo
339488	chr1	36038915-36060929	+	339488	bar
55700	chr1	36621180-36646450	+	55700	boo
5690	chr1	36065142-36107127	-	5690	bar
63967	chr1	36185819-36235579	-	63967	bar
79932	chr1	35859484-36023551	-	79932	bar
9967	chr1	36690033-36770958	+	9967	boo

seqinfo: 25 sequences (1 circular) from hg19 genome

If we want strand specific overlaps, we can add `_directed`:

```
g %>% join_overlap_inner_directed(r)
```

GRanges object with 6 ranges and 2 metadata columns:

	seqnames	ranges	strand	gene_id	range_id
	<Rle>	<IRanges>	<Rle>	<character>	<factor>
105378645	chr1	36035408-36043409	-	105378645	bar
127703	chr1	36179478-36184495	-	127703	bar
27095	chr1	36602171-36621654	-	27095	boo
5690	chr1	36065142-36107127	-	5690	bar
63967	chr1	36185819-36235579	-	63967	bar
79932	chr1	35859484-36023551	-	79932	bar

seqinfo: 25 sequences (1 circular) from hg19 genome

By turning the join around, we have access to the genomic range information about the genes. Now we can compute, e.g. the average genomic extent of the genes (first base to last base), per overlapping range.

```
g %>% join_overlap_inner_directed(r) %>%
  group_by(range_id) %>%
  summarize(count=n(), mean_width=mean(width))
```

DataFrame with 2 rows and 3 columns

	range_id	count	mean_width
	<factor>	<integer>	<numeric>
1	bar	5	53767
2	boo	1	19484

What about "foo"? We need to add a `complete()` call to account for the fact that we are missing those overlaps after the join. We need to call the function explicitly from the *tidyr* package but by not loading the package we can avoid some function name conflicts with *plyranges*. Also we need to convert to tibble (explanation follows).

```
library(tibble)
g %>% join_overlap_inner_directed(r) %>%
  group_by(range_id) %>%
  summarize(count=n(), mean_width=mean(width)) %>%
  as_tibble() %>%
  tidyr::complete(range_id, fill=list(count=0))
```

```
# A tibble: 3 x 3
  range_id count mean_width
  <fct>    <int>    <dbl>
```

1	bar	5	53767
2	boo	1	19484
3	foo	0	NA

Why did we have to convert to tibble before running `complete()`? This is because metadata columns of *GRanges* objects are in a format called *DataFrame*, on which the *tidyr* / *dplyr* functions don't know how to operate.

To access metadata columns of a *GRanges* object, you can use any of these paradigms:

```
mcols(r)
```

DataFrame with 3 rows and 1 column

```
  range_id
<factor>
1      foo
2      bar
3      boo
```

```
mcols(r)$range_id
```

```
[1] foo bar boo
Levels: bar boo foo
```

```
r$range_id # this works also
```

```
[1] foo bar boo
Levels: bar boo foo
```

```
mcols(r)[["range_id"]] # for programmatic access
```

```
[1] foo bar boo
Levels: bar boo foo
```

But if you want to work on them in with *tidyr* / *dplyr*, you need to first convert to tibble (or `data.frame`):

```
mcols(r) %>% as_tibble()
```

```
# A tibble: 3 x 1
  range_id
  <fct>
1 foo
2 bar
3 boo
```

Reduce instead of summarize: Above when we used `group_by` and `summarize` we lost the original range data. Another option, to preserve the range data, is to use the function `reduce_ranges` within groups that we define (which can be `_directed` or not).

If we want to preserve the range information for the `r` object, we can start with `r` and proceed to join, group by, and reduce within groups. For an example of `reduce_ranges` used in the context of a genomic data analysis see Lee, Lawrence, and Love (2020). In order to compute on the gene widths, we have to add that as a metadata column within the join. To keep the no-gene-overlapping ranges in `r`, we can count when the gene ID is not NA.

```
r %>%
  join_overlap_left_directed(g %>% mutate(gene_width=width)) %>%
  group_by(range_id) %>%
  reduce_ranges(count=sum(!is.na(gene_id)),
                mean_width=mean(gene_width))
```

GRanges object with 3 ranges and 3 metadata columns:

	seqnames	ranges	strand	range_id	count	mean_width
	<Rle>	<IRanges>	<Rle>	<factor>	<integer>	<numeric>
[1]	chr1	34000001-34200000	*	foo	0	NA
[2]	chr1	36000001-36200000	*	bar	5	53767
[3]	chr1	36600001-36700000	*	boo	1	19484

seqinfo: 1 sequence from an unspecified genome; no seqlengths

Hopefully, you've seen that there are many routes to compute the types of statistics of interest. The best way to decide which one to use is to think first: what do I want the final output to look like, and do I need to keep track of non-overlapping ranges? This will help dictate the way you set up your code, whether a `join` or a `mutate` to just tally a column of overlaps, etc., and whether a `complete` call is needed to fill in missing levels at the end of the analysis.

4 Gene plots

Objective: for a region of the genome, find peaks near the TSS of genes and then plot their signal strength per gene, stratifying by the tissue origin of the peak.

We start by loading the pre-downloaded peaks ranges:

```
load("data/peaks.rda")
```

Likewise, we want to use hg19 genes again to match the hg19 peaks:

```
library(TxDb.Hsapiens.UCSC.hg19.knownGene)
library(org.Hs.eg.db)
g <- genes(TxDb.Hsapiens.UCSC.hg19.knownGene)
```

Adding gene symbols:

```
library(plyranges)
g <- g %>%
  mutate(gene_name = mapIds(org.Hs.eg.db,
                             gene_id, "SYMBOL", "ENTREZID"))
```

Find a region of the genome near a kidney-specific gene:

```
g %>%
  filter(gene_name == "UMOD")
```

GRanges object with 1 range and 2 metadata columns:

seqnames	ranges	strand	gene_id	gene_name
<Rle>	<IRanges>	<Rle>	<character>	<character>
7369 chr16	20344373-20367623	-	7369	UMOD

seqinfo: 298 sequences (2 circular) from hg19 genome

```
region <- data.frame(
  seqnames="chr16",
  start=20e6,
  end=21e6) %>%
  as_granges()
```

Combine the bladder and kidney peaks, and select certain columns:

```
pks <- bind_ranges(bladder=bladder_pks,
  kidney=kidney_pks,
  .id="tissue") %>%
  select(signal=signalValue, tissue)
```

Finally, we perform the overlap join, locating peaks within 100kb of the TSS of the gene.

```
g_with_pks <- g %>%
  anchor_5p() %>%
  mutate(width=1) %>%
  filter_by_overlaps(region) %>%
  join_overlap_inner(pks, maxgap=1e5)
g_with_pks$tissue %>% table()
```

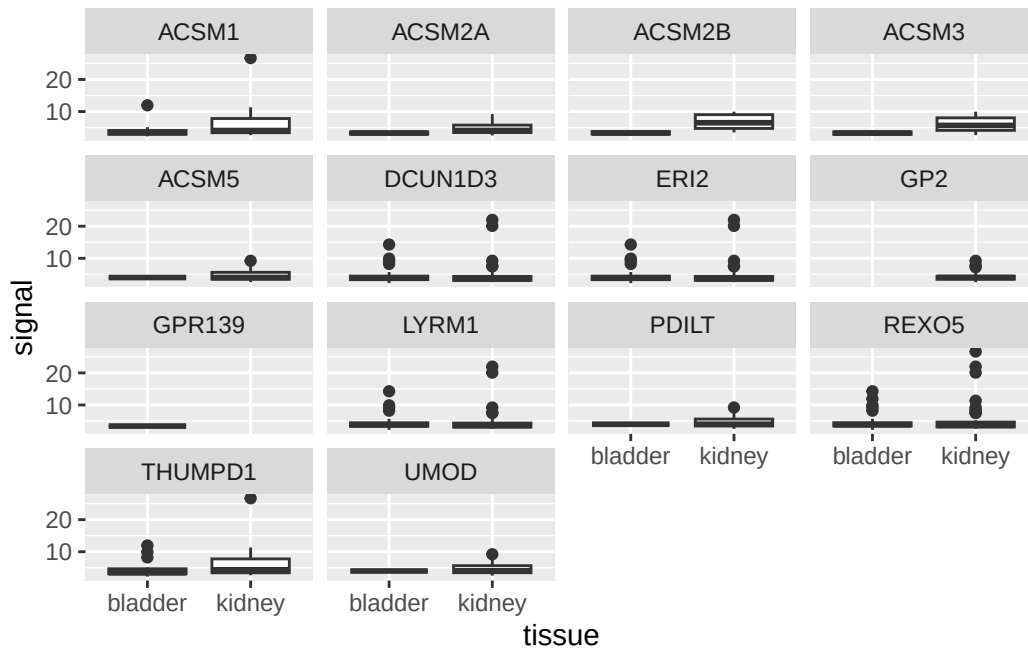
```
.
bladder kidney
    197    328
```

We can construct a faceted set of boxplots, first we make a tibble of data for our plot.

```
library(dplyr)
library(tibble)
dat <- g_with_pks %>%
  select(gene_name, signal, tissue, .drop_ranges=TRUE) %>%
  as_tibble()
```

Then pass the data to ggplot (we could have just passed the data directly, but we plan to re-use the data).

```
library(ggplot2)
dat %>%
  ggplot(aes(tissue, signal)) +
  geom_boxplot() +
  facet_wrap(~gene_name)
```



Now let's try to plot these in context, using *plotgardener* (Kramer et al. 2022). First we filter down to the peaks near *UMOD*.

```
umod <- g %>%
  filter(gene_name == "UMOD") %>%
  anchor_5p() %>%
  mutate(width=1)
pks_to_plot <- pks %>%
  filter_by_overlaps(umod, maxgap=1e5) %>%
  anchor_center() %>%
  mutate(width=1e4) # to make the ranges more visible
```

We then define a color scheme for the tissue variable, and make a ggplot object which will be added to our genome plots.

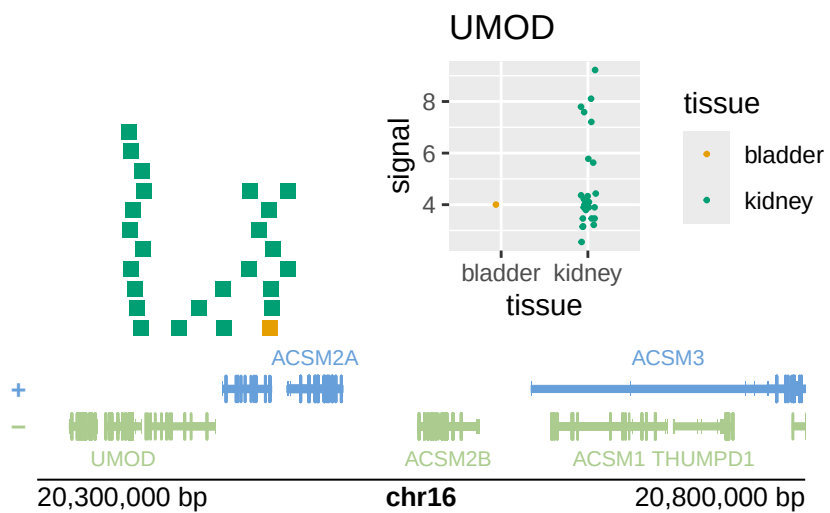
```
cols <- function(n) palette.colors(n+2)[-c(1,3)]
col_vec <- cols(2)
names(col_vec) <- unique(dat$tissue)
p <- dat %>%
  filter(gene_name == "UMOD") %>%
  ggplot(aes(tissue, signal, col=tissue)) +
  # here we set a seed for jitter
  geom_point(size=.5, position=position_jitter(width=.1, seed=5)) +
  scale_color_manual(values = col_vec)
```

We then create some parameters that will be shared across a number of the plots in *plotgardener*.

```
library(plotgardener)
par <- pgParams(
  chrom = "chr16",
  chromstart = round((start(umod) - 1e5)/1e5)*1e5,
  chromend = round((start(umod) + 4e5)/1e5)*1e5,
  assembly = "hg19", just = c("left", "bottom")
)
```

Finally we put all the pieces together on a page (for laying out the plot, first use `showGuides=TRUE`).

```
pageCreate(width = 5, height = 3, showGuides = FALSE)
plotGenes(
  params = par, x = 0.5, y = 2.5, width = 4, height = .75
)
plotRanges(
  pks_to_plot,
  fill = colorby("tissue", palette=cols),
  params = par, x = 0.5, y = 1.75, width = 4, height = 1.75
)
plotGenomeLabel(
  params = par, x = 0.5, y = 2.5, length = 4,
  just = c("left", "top")
)
plotGG(
  p + ggtitle("UMOD"),
  params = par, x = 2.25, y = 1.75, width = 2.5, height = 1.75
)
```



5 Isoform analysis

Objective: compare the structure of isoforms within a gene using grouping and disjoint operations.

There are many packages in Bioconductor that allow for isoform-level analysis across samples (also called transcript-level analysis). Packages that facilitate differential transcript analysis include *DEXSeq* and *DRIMSeq* (demonstrated in the *rnaseqDTU* workflow), and newer packages *satuRn* and *fishpond*.

Once you've identified isoforms of interest within a gene, perhaps isoforms that switch in terms of their expression after cells are treated, one can use the [IsoformSwitchAnalyzeR](#) Bioconductor package to visualize and analyze a set of isoform switches ([Vitting-Seerup and Sandelin 2019](#)). For example, one can test the functional consequences of a set of isoform switches in terms of the gain or loss of protein domains, or splicing-centric changes (e.g. alternative 3' or 5' acceptor sites, alternative transcription start or ends sites, etc.) *IsoformSwitchAnalyzeR* is a multi-feature and mature package for this type of analysis, but we can perform some simpler within-gene isoform comparisons using *plyranges*, mostly for demonstration.

Here we will suppose that we have somehow identified isoforms of interest, and we want to compare these isoforms to other isoforms of the same gene. For simplicity, we will focus on one isoform per gene, for a particular set of interesting genes, just picking isoforms at random for genes on `chr1`. We could use *plyranges* to compare various metadata about isoforms or exons e.g. RNA-seq or ChIP-seq coverage, sequence content, etc. But here we will just compare isoforms alone by the interval definitions. Then to reformulate:

Specific objective 1: compare one isoform per gene to the others, in terms of the extent from TSS to TES. What makes this isoform distinct?

We will start again with the transcript database we've used before:

```
library(TxDb.Hsapiens.UCSC.hg19.knownGene)
txdb <- TxDb.Hsapiens.UCSC.hg19.knownGene
txp <- transcripts(txdb)
```

For further operations it will be convenient to have the `tx_id` be a character variable, and we filter now to a set of genes of interest (here just picking those on `chr1`):

```
library(plyranges)
txp <- txp %>%
  mutate(tx_id = as.character(tx_id)) %>%
  filter(seqnames == "chr1")
```

It is sometimes useful to have other identifiers, such as the gene ID (Entrez)...

```
txp <- txp %>%
  mutate(gene_id = mapIds(
    txdb, keys=tx_id,
    column="GENEID", keytype="TXID")
  ) %>%
  filter(!is.na(gene_id))
```

...and the gene symbol. For simplicity we will keep genes that have a non-NA symbol but this step is not necessary.

```
library(org.Hs.eg.db)
txp <- txp %>%
  mutate(symbol = mapIds(
    org.Hs.eg.db, keys=gene_id,
    column="SYMBOL", keytype="ENTREZID")
  ) %>%
  filter(!is.na(symbol))
```

The following is one way to identify which isoforms belong to multi-isoform genes:

```
# this is slow for some reason...
## txp %>%
##   group_by(gene_id) %>%
##   mutate(ntxp = n()) %>%
##   ungroup()

# this is faster...
txp <- txp %>%
  mutate(ntxp = as.integer(table(txp$gene_id)[ gene_id ]))
```

We can now filter to the multi-isoform genes:

```
txp <- txp %>%
  filter(ntxp > 1)
```

Here we arbitrarily pick one isoform per gene, first by identifying those in a tibble...

```
library(tibble)
set.seed(3)
pick_one <- txp %>%
  as_tibble() %>%
  group_by(gene_id) %>%
  slice(sample.int(n(), size=1)) %>%
  dplyr::pull(tx_id)
```

...then we can label these in our `txp` object. We will track these with an integer, 1 for the isoform of interest, and 0 for the others.

```
txp <- txp %>%
  mutate(the_one = as.integer(tx_id %in% pick_one))
```

To identify which “parts” of the TSS-to-TES interval belong to which isoform, we can use `disjoin_ranges`. This breaks up the ranges, here grouped by gene, into distinct parts, and labels those according to whatever metadata variables we specify. Here we specify to combine `tx_id` into a collapsed string, but we could also perform numeric operations, e.g. `min` or `mean` etc. And we can specify more than one new variable to be created during the `disjoin_ranges` operation. As with `reduce_ranges`, this operation can be `_directed` or not.

```
txp %>%
  group_by(gene_id) %>%
  disjoin_ranges(tx_ids = paste(tx_id, collapse=","))
```

GRanges object with 32079 ranges and 2 metadata columns:

	seqnames	ranges	strand	gene_id
	<Rle>	<IRanges>	<Rle>	<character>
[1]	chr1	11121-11124	*	100287102
[2]	chr1	11125-11409	*	100287102
[3]	chr1	11410	*	100287102
[4]	chr1	11411-11425	*	100287102
[5]	chr1	11426-11769	*	100287102
...	...	...	...	...
[32075]	chr1	249153271	*	55657

```

[32076] chr1 249153272-249153282 * | 55657
[32077] chr1 249153283-249153284 * | 55657
[32078] chr1 249153285-249153315 * | 55657
[32079] chr1 249153316-249153343 * | 55657

```

```

      tx_ids
    <character>
[1]          2
[2]         2,3
[3]        2,3,4
[4]       2,3,4,5
[5]      2,3,4,5,6
...         ...
[32075] 33124,33125,33142,33..
[32076] 33124,33125,33142,33..
[32077] 33124,33125,33143,33..
[32078]          33147,33151
[32079]          33151

```

```
-----
```

```
seqinfo: 298 sequences (2 circular) from hg19 genome
```

Here we try to answer the specific objective, by labeling which parts belong exclusively to the isoform of interest by computing `min(the_one)` (try to convince yourself that this does in fact identify these intervals).

```

txp %>%
  group_by(gene_id) %>%
  disjoin_ranges(the_one_parts = min(the_one)) %>%
  filter(the_one_parts > 0)

```

GRanges object with 751 ranges and 2 metadata columns:

```

      seqnames      ranges strand |      gene_id the_one_parts
      <Rle>        <IRanges> <Rle> | <character>    <integer>
[1]      chr1          991499      * |      375790          1
[2]      chr1    1209413-1210602      * |    101928895          1
[3]      chr1    1227406-1227409      * |         6339          1
[4]      chr1    1263375-1264276      * |         80772          1
[5]      chr1    1355625-1357427      * |    105378585          1
...      ...      ...      ... .      ...      ...
[747]     chr1 238041164-238045703      * |         57829          1
[748]     chr1 240061316-240061318      * |    100873984          1
[749]     chr1 247108849-247130963      * |    100533111          1
[750]     chr1 248841439-248844549      * |         401994          1

```

```
[751]      chr1 248862207-248866067      * |      401994      1
-----
seqinfo: 298 sequences (2 circular) from hg19 genome
```

Do the parts identified make sense if we check one gene?

```
txp %>%
  filter(gene_id == "9651")
```

GRanges object with 8 ranges and 6 metadata columns:

	seqnames	ranges	strand	tx_id	tx_name
	<Rle>	<IRanges>	<Rle>	<character>	<character>
[1]	chr1	2357419-2411243	+	672	ENST00000609981.5_6
[2]	chr1	2398898-2436965	+	673	ENST00000449969.5_13
[3]	chr1	2407728-2436971	+	674	ENST00000378486.8_9
[4]	chr1	2407754-2436964	+	675	ENST00000419816.6_8
[5]	chr1	2410529-2436969	+	676	ENST00000343889.8_2
[6]	chr1	2411327-2436715	+	677	ENST00000278878.6_7
[7]	chr1	2429221-2436967	+	678	ENST00000473964.5_2
[8]	chr1	2430086-2436967	+	679	ENST00000462379.1_2

	gene_id	symbol	ntxp	the_one
	<character>	<character>	<integer>	<integer>
[1]	9651	PLCH2	8	0
[2]	9651	PLCH2	8	0
[3]	9651	PLCH2	8	0
[4]	9651	PLCH2	8	0
[5]	9651	PLCH2	8	0
[6]	9651	PLCH2	8	1
[7]	9651	PLCH2	8	0
[8]	9651	PLCH2	8	0

```
-----
seqinfo: 298 sequences (2 circular) from hg19 genome
```

Let's pause and consider what we've answered so far. We asked, for a given isoform per gene, what parts (intervals) uniquely define that isoform, when we just consider TSS-to-TES extent (ignoring the exonic/intronic structure). We started here mostly for simplicity, but typically we care about *transcribed* sequence, so let's repeat this task, now considering what exonic parts are unique to one isoform per gene.

Specific objective 2: compare one isoform per gene to the others, in terms of the exonic intervals. What makes this isoform distinct?

To start, we will need a list of the exons, grouped by transcript.

```
ebt <- exonsBy(txdb, by="tx")
ebt <- ebt[txp$tx_id] # subset to those txp/genes of interest
```

Here, we could have also used `bind_ranges` but I find that for very large lists of ranges, `unlist` is faster:

```
exons <- unlist(ebt) %>%
  select(exon_id, exon_rank) %>%
  mutate(tx_id = rep(names(ebt), lengths(ebt)))
exons
```

GRanges object with 164866 ranges and 3 metadata columns:

	seqnames	ranges	strand	exon_id	exon_rank	tx_id
	<Rle>	<IRanges>	<Rle>	<integer>	<integer>	<character>
2	chr1	11121-11211	+	3	1	2
2	chr1	12010-12227	+	20	2	2
2	chr1	12613-12721	+	28	3	2
2	chr1	13453-14413	+	40	4	2
3	chr1	11125-11211	+	4	1	3
...	...	...	...	...	...	...
33150	chr1	249151868-249152216	-	80309	3	33150
33151	chr1	249153214-249153343	-	80336	1	33151
33151	chr1	249152064-249152520	-	80315	2	33151
33152	chr1	249153117-249153232	-	80325	1	33152
33152	chr1	249152174-249152584	-	80316	2	33152

seqinfo: 298 sequences (2 circular) from hg19 genome

The `exons` ranges are missing some of our key metadata from `txp`. We can add this, by first `select`-ing what we want from `txp`, turning this into a tibble and `left_join`-ing to the `exons`. I add an `all.equal` step to make sure we have the two tables lined up, before we add the extra columns with `cbind`.

Some of this code wouldn't be necessary for TranscriptDb with more details `exons` output, as with *ensembl*.

```
txp_data <- txp %>%
  select(tx_id, gene_id, ntxp, the_one, .drop_ranges=TRUE) %>%
  as_tibble()
ids <- dplyr::left_join(tibble(tx_id = exons$tx_id),
  txp_data, by="tx_id")
ids
```

```
# A tibble: 164,866 x 4
  tx_id gene_id   ntxp the_one
  <chr> <chr>   <int>   <int>
1 2     100287102    26     0
2 2     100287102    26     0
3 2     100287102    26     0
4 2     100287102    26     0
5 3     100287102    26     0
6 3     100287102    26     0
7 3     100287102    26     0
8 3     100287102    26     0
9 4     100287102    26     0
10 4     100287102    26     0
# i 164,856 more rows
```

```
all.equal(exons$tx_id, ids$tx_id)
```

```
[1] TRUE
```

```
mccols(exons) <- cbind(mcols(exons), ids %>% select(-tx_id))
exons
```

GRanges object with 164866 ranges and 6 metadata columns:

	seqnames	ranges	strand	exon_id	exon_rank	tx_id
	<Rle>	<IRanges>	<Rle>	<integer>	<integer>	<character>
2	chr1	11121-11211	+	3	1	2
2	chr1	12010-12227	+	20	2	2
2	chr1	12613-12721	+	28	3	2
2	chr1	13453-14413	+	40	4	2
3	chr1	11125-11211	+	4	1	3
...	...	...	...	...	...	...
33150	chr1	249151868-249152216	-	80309	3	33150
33151	chr1	249153214-249153343	-	80336	1	33151
33151	chr1	249152064-249152520	-	80315	2	33151
33152	chr1	249153117-249153232	-	80325	1	33152
33152	chr1	249152174-249152584	-	80316	2	33152

	gene_id	ntxp	the_one
	<character>	<integer>	<integer>
2	100287102	26	0
2	100287102	26	0
2	100287102	26	0

```

      2  100287102      26      0
      3  100287102      26      0
      ...      ...      ...      ...
33150      55657      30      0
33151      55657      30      0
33151      55657      30      0
33152      55657      30      0
33152      55657      30      0
-----
seqinfo: 298 sequences (2 circular) from hg19 genome

```

We repeat similar code as performed above with `txp`, now identifying parts of exons that are unique to the isoform of interest, per gene:

```

exon_parts <- exons %>%
  group_by(gene_id) %>%
  disjoin_ranges(the_one_parts = min(the_one)) %>%
  filter(the_one_parts > 0)
exon_parts

```

GRanges object with 2509 ranges and 2 metadata columns:

```

      seqnames      ranges strand |      gene_id the_one_parts
      <Rle>      <IRanges> <Rle> | <character>      <integer>
[1]      chr1      11770-11818      * |      100287102          1
[2]      chr1      898932-899299      * |      339451          1
[3]      chr1      948877-948956      * |      9636          1
[4]      chr1      991499          * |      375790          1
[5]      chr1      1109596-1109645      * |      254173          1
...      ...      ...      ...      ...
[2505]      chr1 247803150-247803301      * |      102724446          1
[2506]      chr1 248841439-248844549      * |      401994          1
[2507]      chr1 248865212-248865306      * |      401994          1
[2508]      chr1 248866004-248866067      * |      401994          1
[2509]      chr1 249107362-249108372      * |      80851          1
-----
seqinfo: 298 sequences (2 circular) from hg19 genome

```

To confirm that we've identified the right parts, let's use *plotgardener* to visualize a particular gene:

```
txp %>%
  filter(gene_id == "339451")
```

GRanges object with 4 ranges and 6 metadata columns:

	seqnames	ranges	strand	tx_id	tx_name
	<Rle>	<IRanges>	<Rle>	<character>	<character>
[1]	chr1	895964-901099	+	288	ENST00000338591.8_4
[2]	chr1	896829-897858	+	289	ENST00000463212.1_1
[3]	chr1	898107-899910	+	290	ENST00000466300.1_1
[4]	chr1	898932-899544	+	291	ENST00000481067.1_1

	gene_id	symbol	ntxp	the_one
	<character>	<character>	<integer>	<integer>
[1]	339451	KLHL17	4	0
[2]	339451	KLHL17	4	0
[3]	339451	KLHL17	4	0
[4]	339451	KLHL17	4	1

seqinfo: 298 sequences (2 circular) from hg19 genome

```
tx_to_show <- txp %>%
  filter(gene_id == "339451" & the_one == 1) %>%
  as_tibble() %>%
  dplyr::pull(tx_name)
these_parts <- exon_parts %>%
  filter(gene_id == "339451")
```

We lay out a page zooming into this gene and its isoforms:

```
library(plotgardener)
par <- pgParams(
  chrom = "chr1",
  chromstart = 895.9e3, chromend = 901.2e3,
  assembly = "hg19", just = c("left", "bottom")
)
```

We will highlight our isoform of interest:

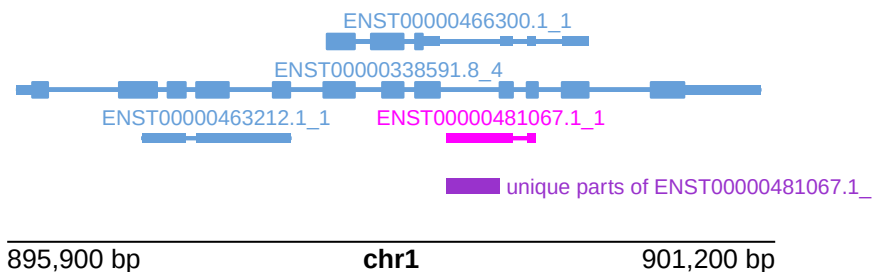
```
hilite <- data.frame(transcript=tx_to_show, color="magenta")
```

Finally, putting it all together:

```

pageCreate(width = 5, height = 2.5, showGuides = FALSE)
plotTranscripts(
  params = par, x = 0.5, y = 1.5, width = 4, height = 1.5,
  transcriptHighlights = hilite
)
plotRanges(
  these_parts, fill="darkorchid",
  params = par, x = 0.5, y = 1.75, width = 4, height = .25
)
label <- paste("unique parts of", tx_to_show)
plotText(
  label = label, fontcolor = "darkorchid",
  params = par, x = 3.1, y = 1.75,
  just = c("left", "bottom"), fontsize = 8
)
plotGenomeLabel(
  params = par, x = 0.5, y = 2, length = 4,
  just = c("left", "top")
)

```



Questions:

- How else could we have found parts of one isoform per gene, that do not belong to any other isoforms of the genes. Would other approaches have any limitations?

6 Join is an overlap

Objective: learn that a `join` operation implies finding overlapping data in two tables, according to some particular criterion.

We start with a quick example explaining why we use functions called `join`. Later we will extend from simple tables to examples making use of genomic ranges.

```
library(dplyr)
library(tidyr)
library(randomNames)
```

Let's set up a simulation where we have 17 classmates (not counting ourselves) in a school of 118. Suppose every member of the class has 12 friends outside of class.

We want to find out, for each classmate, how many friends we have in common and also, of our shared friends, what is a typical birthday. Let's define typical as the median birthday of our shared friends.

```
m <- 17 # size of class
n <- 100 # size of others in school
nf <- 12 # number of friends outside class
set.seed(5)
people <- randomNames(m + n,
                      name.order="first.last",
                      name.sep=" ")
```

We define the population of potential friends (those outside the class) as `pop`, and our 12 friends are in `friends`. Finally, we define the people that are in our class as `class_people`.

```
pop <- data.frame(person=people[-(1:m)],
                  birthday=sample(31, n, TRUE))
friends <- data.frame(friend=sample(people, nf))
class_people <- people[1:m]
```

The following sets up a `data.frame`, where each row gives, for a given classmate, their friends, and the friends' birthdays.

```

loop <- rep(1:m, each=nf)
idx <- as.vector(replicate(m, sample(n, nf)))
class <- data.frame(classmate=class_people[loop],
                    friend=pop$person[idx],
                    birthday=pop$birthday[idx])

```

Sort this by classmate alphabetically, and make classmate into a factor. This last step is important, as it will help us to keep track of the classmates for whom we share *no* friends outside of class.

```

class <- class %>%
  arrange(classmate) %>%
  mutate(classmate=factor(classmate))

```

We perform an `inner_join` by "friend". This just means we look for classmates where we have overlapping friends, and we drop the rows where we don't share any friends. "Inner" refers to the fact that we are keeping the overlap in the *inside* of two intersecting circles. Note that the join operation brings along the metadata (extra data) about the friends' birthdays.

```

shared_friends <- friends %>%
  inner_join(class, by="friend") %>%
  arrange(classmate, friend) %>%
  relocate(classmate) # classmate to 1st column
shared_friends

```

	classmate	friend	birthday
1	Brandon Jones	Christopher Rivera	13
2	Brandon Jones	Jose Jimenez	26
3	Brandon Jones	Maria Hernandez	15
4	Brandon Jones	Tuli Hoang	7
5	Christopher Herron	Tuli Hoang	7
6	Christopher Herron	William Steinbach	15
7	Haley Polhamus	Cellene Millhone	23
8	Haley Polhamus	William Steinbach	15
9	Juan Villegas Cabrera	Alyssa Kinanee	30
10	Juan Villegas Cabrera	Malik Gammon	5
11	Juan Villegas Cabrera	Tajhae Bohanna	31
12	Maisara el-Arshad	Kaylyn Judkins	2
13	Maisara el-Arshad	Maria Hernandez	15
14	Maisara el-Arshad	Tajhae Bohanna	31
15	Michael McGill	Christopher Rivera	13

16	Michael McGill	Maria Hernandez	15
17	Noah Pettinger	Kaylyn Judkins	2
18	Noah Pettinger	Tajhae Bohanna	31
19	Orion Villani	Malik Gammon	5
20	Shane Ranaldi	Alyssa Kinanee	30
21	Shane Ranaldi	William Steinbach	15
22	Stephanie Hernandez	Asmaa el-Abdelnour	29
23	Zachary Roe-Huffman	Maria Hernandez	15

Lastly, we perform some summarization: compute the number of friends in common with `n()` and the median birthday of shared friends. The `complete` call at the end fills in 0 for those classmates for whom we share no friends (here, the use of `factor` earlier becomes relevant). We can choose which columns to fill in, and what value to add.

```
shared_friends %>%
  group_by(classmate) %>%
  summarize(in_common=n(), median_bday=median(birthday)) %>%
  complete(classmate, fill=list(in_common=0,median_bday=-1))
```

```
# A tibble: 17 x 3
  classmate      in_common median_bday
  <fct>          <int>      <dbl>
1 Aaliyah Minter         0         -1
2 Brandon Jones         4         14
3 Christopher Herron     2         11
4 Collin Leon           0         -1
5 Haley Polhamus        2         19
6 Juan Villegas Cabrera  3         30
7 Khaalid el-Ammar      0         -1
8 Kianna Mcalevy        0         -1
9 Maazin al-Ismael      0         -1
10 Maisara el-Arshad     3         15
11 Michael McGill       2         14
12 Noah Pettinger       2        16.5
13 Orion Villani        1          5
14 Shane Ranaldi        2        22.5
15 Stephanie Hernandez   1         29
16 Xavier Urueta        0         -1
17 Zachary Roe-Huffman   1         15
```

7 Many genomic models

Objective: learn how to run many models (think many linear regressions or many machine learning models) in a tidy framework, by “nesting” the genomic dataset and storing the fitted models as rows of a new column in the nested table.

In the previous chapter, we looked at some basic filtering and plotting operations, with perhaps the most interesting operation being a grouped centering of the gene expression. Here we will explore a special case of tidy-style operations on a genomic dataset, in particular a *SummarizedExperiment*, where we want to run multiple, similar models across groups of features (or likewise, the same could be done to rows).

For some other references to these types of operations, you can check out:

- The “many models” chapter of R for Data Science by Wickham and Golemund (2017), which introduced the basics of how to run many models in a tidy framework
- The [nest](#) documentation in *tidyr*
- The [tidymodels](#) package

Given the size of genomic data, and the way in which *tidySummarizedExperiment* abstracts the link between data (`assay`) and metadata (`rowData` and `colData`), we will consider in this chapter how the speed of the operations may be impacted by our choices in setting up the code. In the end, you may find that a more standard way of running the analyses (e.g. using base R/Bioc) is more efficient, depending on the cost incurred by the nesting operation, which we will describe shortly.

We will work with a Bioconductor experiment package *fission* which contains a dataset created by Leong et al. (2014) [PMC4050258](#):

Here we integrate high-throughput RNA sequencing and label-free quantitative protein mass spectrometry to investigate global changes in transcript and protein levels in the fission yeast stress response.

The experiment considers fission yeast of two strains, wild-type (WT) and *atf21Δ*, but here we will not focus on these two strains but just lump the data together, to increase our sample size for the point of building the models.

```
library(fission)
library(magrittr)
data(fission)
se <- fission
colData(se)
```

DataFrame with 36 rows and 4 columns

	strain	minute	replicate	id
	<factor>	<factor>	<factor>	<character>
GSM1368273	wt	0	r1	wt_0_r1
GSM1368274	wt	0	r2	wt_0_r2
GSM1368275	wt	0	r3	wt_0_r3
GSM1368276	wt	15	r1	wt_15_r1
GSM1368277	wt	15	r2	wt_15_r2
...	...	...	...	...
GSM1368304	mut	120	r2	mut_120_r2
GSM1368305	mut	120	r3	mut_120_r3
GSM1368306	mut	180	r1	mut_180_r1
GSM1368307	mut	180	r2	mut_180_r2
GSM1368308	mut	180	r3	mut_180_r3

We again use *tidybulk* to filter to abundant genes and scale counts for library size:

```
library(tidybulk)
se <- se %>%
  keep_abundant(factor_of_interest = strain) %>%
  scale_abundance()
```

Warning: The `factor\_of\_interest` argument of `keep\_abundant()` is deprecated as of tidybulk 2.0.0.

i Please use the `formula\_design` argument instead.

i The argument 'factor_of_interest' is deprecated and will be removed in a future release. Please use the 'design' or 'formula_design' argument instead.

Warning: The `factor\_of\_interest` argument of `identify\_abundant()` is deprecated as of tidybulk 2.0.0.

i Please use the `formula\_design` argument instead.

i The argument 'factor_of_interest' is deprecated and will be removed in a future release. Please use the 'design' or 'formula_design' argument instead.

i The deprecated feature was likely used in the tidybulk package.

Please report the issue at <<https://github.com/stemangiola/tidybulk/issues>>.

```
assayNames(se)
```

```
[1] "counts"          "counts_scaled"
```

We make a PCA plot of the log scaled counts, to get a sense for how the samples vary. Note that minute 0 and 180 are similar, as the cells have not yet responded to the stimulus at minute 0. We will later remove these samples for simple modeling.

```
pca <- se %>%  
  reduce_dimensions(method="PCA")
```

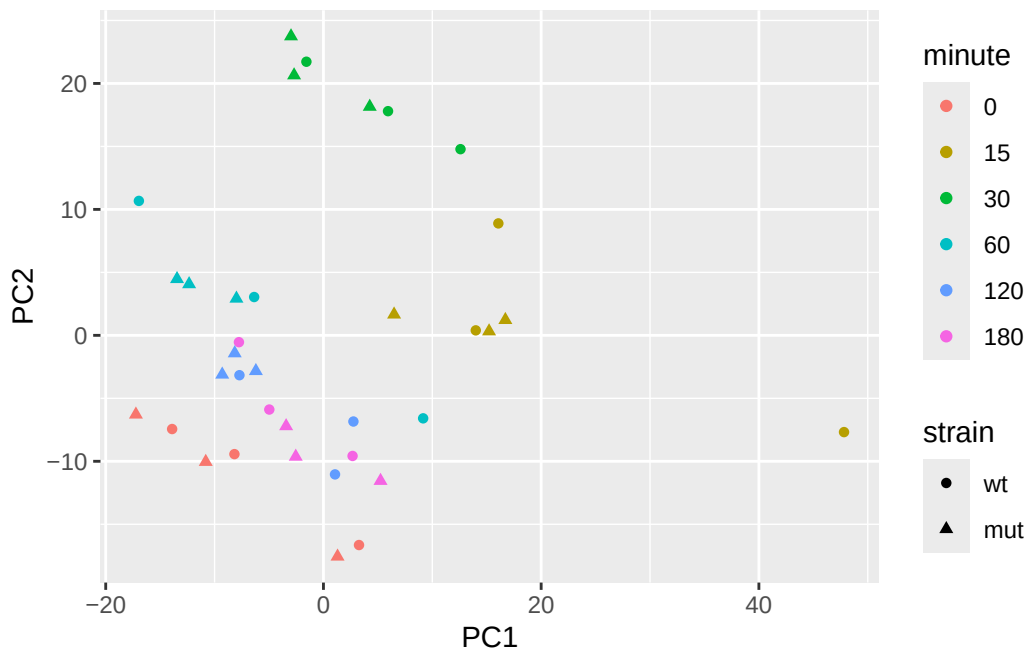
Getting the 500 most variable genes

Fraction of variance explained by the selected principal components

```
# A tibble: 2 x 2  
  `Fraction of variance`    PC  
          <dbl> <int>  
1          0.0645     1  
2          0.0495     2
```

tidybulk says: to access the raw results do ``metadata(.)$tidybulk$PCA``

```
library(ggplot2)  
pca %>%  
  pivot_sample() %>%  
  ggplot(aes(PC1, PC2, shape=strain, color=minute)) +  
  geom_point()
```



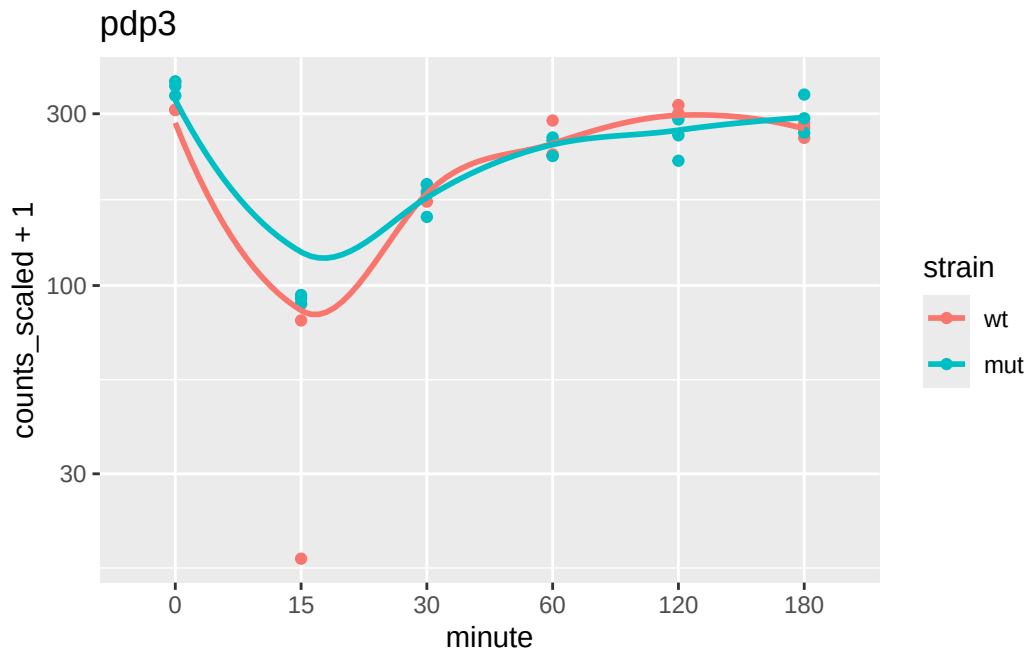
We can plot the gene with the most contribution to PC1. Here we will begin working with the data as a *tidySE* (shortened name for *tidySummarizedExperiment*):

```
max_pc1 <- which.max(abs(metadata(pca)[["tidybulk"]][["PCA"]][["rotation"]][, "PC1"]))
max_pc1
```

```
SPAC23D3.01
304
```

```
library(tidySummarizedExperiment)
se %>%
  filter(.feature == names(max_pc1)) %>%
  ggplot(aes(minute, counts_scaled + 1, color=strain, group=strain)) +
  geom_point() +
  stat_smooth(se=FALSE) +
  scale_y_log10() +
  ggtitle( rowData(se)[names(max_pc1), "symbol"] )
```

Warning: `when()` was deprecated in purrr 1.0.0.
 i Please use `if` instead.
 i The deprecated feature was likely used in the tidySummarizedExperiment package.
 Please report the issue at
<https://github.com/stemangiola/tidySummarizedExperiment/issues>.



Now, let's consider a hypothetical analysis question: can we predict the **minute** (quantitatively) using the log gene expression of a genes in a neighborhood on the chromosome. While this is mostly a contrived question, for the purposes of demonstrating nesting of genomic datasets, you could imagine that there might be modules of responsive genes positioned along the chromosome, and that a prediction task is one way to identify genes that are related to an aspect of the experimental design.

The steps will be:

- Create centered log scaled counts
- Create “blocks” of genes, by tiling the genome and labeling the genes that fall within the same tile
- “Nest” the tidySE such that we can operate on the blocks of genes together
- Run a series of models, each time predicting the **minute** variable using the expression of the genes in the block
- Evaluate these models (here simply looking at in-sample training error)

“Nesting” a dataset is an operation, similar to **group_by**, where a variable is used to perform grouped operations. We will specify to nest all the data (columns) besides the grouping variable, such that we end up with a *tibble* that looks like:

grouping variable	data
value1	RngdSmmE
value2	RngdSmmE

...

Hence, for every row of the `SummarizedExperiment` that has `value1` for the grouping variable, we will have a subsetting `SummarizedExperiment` (“ranged” refers to the fact that it is `rowRanges`).

Let’s start with the first task. We compute `logcounts` and then center and scale these values. Likewise, we turn the `minute` variable from a factor into a numeric, and scale from 0 to 1. These changes would help us compare coefficients across gene later.

```
se <- se %>%
  mutate(logcounts = log2(counts_scaled + 1),
         logcounts = (logcounts - mean(logcounts))/sd(logcounts))
se <- se %>%
  mutate(time = as.numeric(as.character(minute)) / 180)
se
```

```
class: RangedSummarizedExperiment
dim: 5861 36
metadata(3): ' tidybulk latest_mutate_scope_report
assays(3): counts counts_scaled logcounts
rownames(5861): SPAC212.09c SPAC212.04c ... SPMITTRNAGLU.01 SPMIT.11
rowData names(3): symbol biotype .abundant
colnames(36): GSM1368273 GSM1368274 ... GSM1368307 GSM1368308
colData names(7): strain minute ... multiplier time
```

For demonstration, we will work with just the first chromosome: I.

For our task of modeling the design using gene expression, in blocks along the genome, we need to create tiles to determine which genes to group together. To do so, we need to know how long the chromosomes are. The original [publication](#) states:

Sequencing reads were aligned to the fission yeast genome (PomBase database release 11)

Usually we would look for the length of chromosomes from a source that hosts the reference (e.g. UCSC genome lengths can be obtained using `Seqinfo`). In this case, I wasn’t able to find information about this particular release, so I just guess the length of the chromosome using the gene with the largest coordinate:

```
library(plyranges)
rowRanges(se) %>%
  filter(seqnames == "I") %>%
  summarize(max(end))
```

```
DataFrame with 1 row and 1 column
  max.end.
<integer>
1  5556768
```

```
genes_to_keep <- rowRanges(se) %>%
  filter(seqnames == "I") %>%
  names()
```

We now filter the `se` object to remove the 0 time point, and to keep just the features on chromosome I.

```
se0 <- se # save the original object
se <- se %>%
  filter(time != 0) %>%
  filter(.feature %in% genes_to_keep)
```

To make tiles on chromosome I, we just need to specify the extent (here I plug in the largest gene coordinate):

```
tiles <- data.frame(seqnames="I",start=1,end=5.6e6) %>%
  as_granges() %>%
  tile_ranges(width=1e5) %>%
  select(-partition) %>%
  mutate(tile = seq_along(.))
tiles
```

GRanges object with 56 ranges and 1 metadata column:

	seqnames	ranges	strand	tile
	<Rle>	<IRanges>	<Rle>	<integer>
[1]	I	1-100000	*	1
[2]	I	100001-200000	*	2
[3]	I	200001-300000	*	3
[4]	I	300001-400000	*	4
[5]	I	400001-500000	*	5
...	...	...	...	...
[52]	I	5100001-5200000	*	52
[53]	I	5200001-5300000	*	53
[54]	I	5300001-5400000	*	54
[55]	I	5400001-5500000	*	55
[56]	I	5500001-5600000	*	56

```
seqinfo: 1 sequence from an unspecified genome; no seqlengths
```

We now determine which tile the genes fall in (using TSS only, so that genes fall in a single tile). We can add this data back onto the tidySE using a `left_join`:

```
ranges_tiled <- rowRanges(se) %>%
  anchor_5p() %>%
  mutate(width=1) %>%
  join_overlap_left(tiles) %>%
  mutate(.feature = names(.)) %>%
  select(tile, .feature, .drop_ranges=TRUE) %>%
  as_tibble()
nrow(se) == nrow(ranges_tiled)
```

```
[1] TRUE
```

```
# combining the tile information with the SE
se <- se %>% left_join(ranges_tiled)
```

Joining with ``by = join_by(.feature)``

Typically we have a little less than 50 genes per tile:

```
summary(as.vector(table(rowData(se)$tile)))
```

Min.	1st Qu.	Median	Mean	3rd Qu.	Max.
5.00	44.00	49.00	46.98	51.25	59.00

Next, we want to create a *nested* table, where tidySE objects are grouped by tile and placed within a column of the table. There are a few choices on how to proceed. One option would be to `pivot_wider` the tidySE, as in this chunk below:

```
se %>%
  filter(.feature %in% rownames(se)[1:5]) %>%
  select(.sample, strain, time, .feature, logcounts) %>%
  pivot_wider(names_from = .feature, values_from = logcounts)
```

`tidySummarizedExperiment` says: A data frame is returned for independent data analysis.

```
# A tibble: 30 x 8
  .sample strain   time SPAC212.09c SPAC212.04c SPAC977.11 SPAC977.13c
  <chr>   <fct>   <dbl>   <dbl>   <dbl>   <dbl>   <dbl>
1 GSM1368276 wt     0.0833   -1.30   -1.51   -0.707   -1.85
2 GSM1368277 wt     0.0833   -1.03   -2.01   -1.20   -1.27
3 GSM1368278 wt     0.0833   -0.994  -2.11   -1.14   -1.88
4 GSM1368279 wt     0.167    -0.804  -1.40   -0.692  -0.325
5 GSM1368280 wt     0.167    -0.576  -1.06   -0.849  -0.401
6 GSM1368281 wt     0.167    -0.687  -0.881  -0.687  -0.572
7 GSM1368282 wt     0.333    -0.746  -1.56   -1.10   -1.45
8 GSM1368283 wt     0.333    -0.960  -1.58   -0.912  -0.830
9 GSM1368284 wt     0.333    -0.833  -1.51   -0.875  -1.07
10 GSM1368285 wt     0.667    -1.03   -1.49   -0.868  -1.74
# i 20 more rows
# i 1 more variable: SPAC977.15 <dbl>
```

This ends up being a bit slower than just extracting the information with `assay` and transposing it.

First let's create our nested dataset:

```
library(purrr)
nested <- se %>%
  nest(data = ~tile)
nested
```

```
# A tibble: 56 x 2
  tile data
  <int> <list>
1     1 <RngdSmmE[,30]>
2     2 <RngdSmmE[,30]>
3     3 <RngdSmmE[,30]>
4     4 <RngdSmmE[,30]>
5     5 <RngdSmmE[,30]>
6     6 <RngdSmmE[,30]>
7     7 <RngdSmmE[,30]>
8     8 <RngdSmmE[,30]>
9     9 <RngdSmmE[,30]>
10    10 <RngdSmmE[,30]>
# i 46 more rows
```

Now, by row, extract out and transpose log scaled counts:

```
nested <- nested %>%
  mutate(trainx = map(data, \(d) {
    t(assay(d, "logcounts"))
  }))
nested
```

```
# A tibble: 56 x 3
   tile data          trainx
   <int> <list>        <list>
1     1 1 <RngdSmmE[,30]> <dbl [30 x 20]>
2     2 2 <RngdSmmE[,30]> <dbl [30 x 54]>
3     3 3 <RngdSmmE[,30]> <dbl [30 x 41]>
4     4 4 <RngdSmmE[,30]> <dbl [30 x 54]>
5     5 5 <RngdSmmE[,30]> <dbl [30 x 44]>
6     6 6 <RngdSmmE[,30]> <dbl [30 x 53]>
7     7 7 <RngdSmmE[,30]> <dbl [30 x 45]>
8     8 8 <RngdSmmE[,30]> <dbl [30 x 47]>
9     9 9 <RngdSmmE[,30]> <dbl [30 x 41]>
10    10 10 <RngdSmmE[,30]> <dbl [30 x 59]>
# i 46 more rows
```

We fit an elastic net model ([Friedman, Hastie, and Tibshirani 2010](#)).

```
library(glmnet)
y <- colData(se)$time
nested <- nested %>%
  mutate(fit = map(trainx, \(x) {
    glmnet(x = x, y = y, alpha = .5, lambda = .1)
  }))
nested
```

```
# A tibble: 56 x 4
   tile data          trainx          fit
   <int> <list>        <list>        <list>
1     1 1 <RngdSmmE[,30]> <dbl [30 x 20]> <elnet>
2     2 2 <RngdSmmE[,30]> <dbl [30 x 54]> <elnet>
3     3 3 <RngdSmmE[,30]> <dbl [30 x 41]> <elnet>
4     4 4 <RngdSmmE[,30]> <dbl [30 x 54]> <elnet>
5     5 5 <RngdSmmE[,30]> <dbl [30 x 44]> <elnet>
6     6 6 <RngdSmmE[,30]> <dbl [30 x 53]> <elnet>
7     7 7 <RngdSmmE[,30]> <dbl [30 x 45]> <elnet>
```

```

8      8 <RngdSmmE[,30]> <dbl [30 x 47]> <elnet>
9      9 <RngdSmmE[,30]> <dbl [30 x 41]> <elnet>
10     10 <RngdSmmE[,30]> <dbl [30 x 59]> <elnet>
# i 46 more rows

```

We use the elastic net model, to predict the design from the gene expression (a variable number and set of genes per tile):

```

nested <- nested %>%
  mutate(
    pred = map2(trainx, fit, \(tr,fit) {
      predict(fit, newx = tr)[,1]
    }),
    n = map_dbl(data, nrow),
    in_r2 = map_dbl(pred, \(pred) cor(pred,y)^2)
  )
nested

```

```

# A tibble: 56 x 7
   tile data          trainx          fit      pred      n in_r2
  <int> <list>         <list>         <list> <list>    <dbl> <dbl>
1     1 1 <RngdSmmE[,30]> <dbl [30 x 20]> <elnet> <dbl [30]>  20 0.852
2     2 2 <RngdSmmE[,30]> <dbl [30 x 54]> <elnet> <dbl [30]>  54 0.908
3     3 3 <RngdSmmE[,30]> <dbl [30 x 41]> <elnet> <dbl [30]>  41 0.906
4     4 4 <RngdSmmE[,30]> <dbl [30 x 54]> <elnet> <dbl [30]>  54 0.880
5     5 5 <RngdSmmE[,30]> <dbl [30 x 44]> <elnet> <dbl [30]>  44 0.912
6     6 6 <RngdSmmE[,30]> <dbl [30 x 53]> <elnet> <dbl [30]>  53 0.892
7     7 7 <RngdSmmE[,30]> <dbl [30 x 45]> <elnet> <dbl [30]>  45 0.894
8     8 8 <RngdSmmE[,30]> <dbl [30 x 47]> <elnet> <dbl [30]>  47 0.857
9     9 9 <RngdSmmE[,30]> <dbl [30 x 41]> <elnet> <dbl [30]>  41 0.890
10    10 10 <RngdSmmE[,30]> <dbl [30 x 59]> <elnet> <dbl [30]>  59 0.894
# i 46 more rows

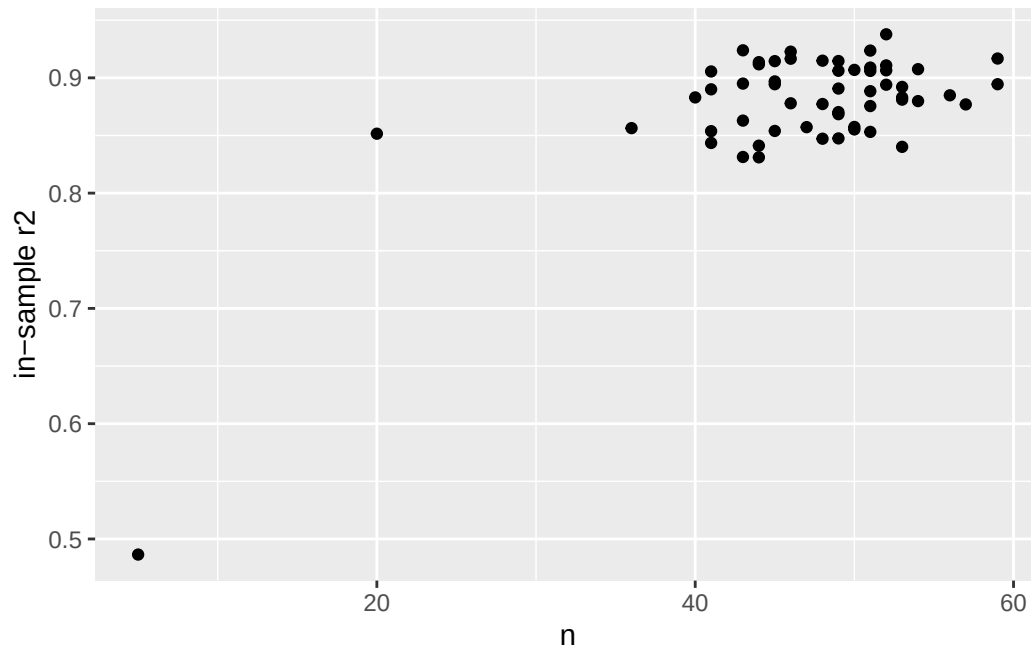
```

Finally we plot the prediction R^2 , and compare to the number of genes in the models:

```

library(ggplot2)
ggplot(nested, aes(n, in_r2)) +
  geom_point() +
  ylab("in-sample r2")

```



Questions:

- How else could we have performed the analysis, without doing the nesting operation? Would this have been faster? What other variables would need to be created to keep track of the many models?
- What advantages or disadvantages can you think about for the different ways of running multiple models across large genomic datasets?

8 Promoter marks

Objective: determine if tissue-specific promoter marks (e.g. H3K27ac) are often near genes that are expressed in a tissue-specific manner.

We will load expression data from the GTEx project ([Lonsdale et al. 2013](#)), which gives median expression in TPM for each tissue. We will use H3K27ac ChIP-seq data from the ENCODE project ([Dunham et al. 2012](#)).

```
library(tidyr)
file <- "data/GTEx_Analysis_2017-06-05_v8_RNASeQCv1.1.9_gene_median_tpm.gct.gz"
gtex <- read.delim(file, skip=2)
```

We select two tissues, bladder and kidney, and convert the data from a wide format into a tidy format.

```
tissues <- gtex %>% dplyr::select(Name, Bladder, Kidney...Cortex) %>%
  dplyr::rename(gene = Name, Kidney = Kidney...Cortex) %>%
  dplyr::mutate(gene = sub("\\.\\.*", "", gene)) %>%
  pivot_longer(!gene, names_to="tissue", values_to="tpm")
```

Now define two vectors of genes that are specific to bladder and kidney:

```
bladder_expr <- tissues %>%
  dplyr::filter(tissue == "Bladder" & tpm > 10) %>%
  dplyr::pull(gene)
kidney_expr <- tissues %>%
  dplyr::filter(tissue == "Kidney" & tpm > 10) %>%
  dplyr::pull(gene)
int <- intersect(bladder_expr, kidney_expr)
bladder_expr <- setdiff(bladder_expr, int)
kidney_expr <- setdiff(kidney_expr, int)
# save(bladder_expr, kidney_expr, file="data/bladder_kidney_expr.rda")
```

Next, use an existing TxDb to locate these genes in the genomes. While we usually recommend to use GENCODE genes for human analysis, because the ENCODE chromatin modification

peak files on AnnotationHub are in hg19, we use the UCSC hg19 genes here for simplicity of the code:

```
library(TxDb.Hsapiens.UCSC.hg19.knownGene)
library(org.Hs.eg.db)
```

Add the ENSEMBL ID and pull out the two tissue-specific sets.

```
g <- genes(TxDb.Hsapiens.UCSC.hg19.knownGene)
```

24 genes were dropped because they have exons located on both strands of the same reference sequence or on more than one reference sequence, so cannot be represented by a single genomic range.

Use 'single.strand.genes.only=FALSE' to get all the genes in a GRangesList object, or use suppressMessages() to suppress this message.

```
library(plyranges)
g <- g %>% mutate(ensembl = mapIds(org.Hs.eg.db, gene_id, "ENSEMBL", "ENTREZID"))
bladder_g <- g %>% filter(ensembl %in% bladder_expr)
kidney_g <- g %>% filter(ensembl %in% kidney_expr)
```

Finally we combine the two sets with `bind_ranges`, and we change the feature size from the whole gene extent (the range from the leftmost exon to rightmost exon), to just the TSS, using `anchor_5p` and `mutate`.

```
tss <- bind_ranges(bladder=bladder_g,
                  kidney=kidney_g,
                  .id="gtissue") %>%
  anchor_5p() %>%
  mutate(width=1)
```

Now we will obtain the H3K27ac peak sets:

```
library(AnnotationHub)
ah <- AnnotationHub()
# query(ah, c("Homo sapiens", "bladder", "H3K27ac", "narrowPeak"))
bladder_pks <- ah[["AH44180"]]
# query(ah, c("Homo sapiens", "kidney", "H3K27ac", "narrowPeak"))
kidney_pks <- ah[["AH43443"]]
save(bladder_pks, kidney_pks, file="data/peaks.rda")
```

We download these and scale so they have the same 90% quantile of `signalValue`.

```
ninety <- function(x) quantile(x, .9, names=FALSE)
bladder_pks <- bladder_pks %>%
  mutate(signal = signalValue / ninety(signalValue))
kidney_pks <- kidney_pks %>%
  mutate(signal = signalValue / ninety(signalValue))
```

Combine the peaks from bladder and kidney, filter to those with $< 0.1\%$ FDR, and center the peak on the summit (the `peak` column gives the shift from the left side to the summit).

```
pks <- bind_ranges(bladder=bladder_pks,
                  kidney=kidney_pks,
                  .id="ptissue") %>%
  filter(qValue > 3, width <= 1000) %>%
  mutate(start = start + peak) %>%
  select(-peak) %>%
  mutate(width = 1)
```

Finally, once we have two tidy range sets, we can perform the analysis by a join, followed by two lines that take care of multiple overlaps, followed by two lines that give us our tallies of interest.

It appears that tissue-specific peaks are enriched near the tissue-specific genes for both bladder and kidney.

```
tss %>%
  join_overlap_left(pks, maxgap=500) %>%
  group_by(ptissue) %>% # within peak tissue...
  filter(!duplicated(gene_id)) %>% # ...just take the first overlap per gene
  group_by(gtissue, ptissue) %>%
  summarize(count = n())
```

DataFrame with 6 rows and 3 columns

	gtissue	ptissue	count
	<Rle>	<character>	<integer>
1	bladder	bladder	2055
2	bladder	kidney	1496
3	bladder	NA	997
4	kidney	bladder	97
5	kidney	kidney	192
6	kidney	NA	279

The above number could also be found with four `countOverlaps` calls, by considering all four pairs of overlaps of the two sets of genes and peaks.

Another way to avoid counting overlaps more than once per gene is to use the *plyranges* function, `n_distinct()`:

```
tss %>%
  join_overlap_left(pks, maxgap=500) %>%
  group_by(gtissue, ptissue) %>%
  summarize(count = n_distinct(gene_id))
```

DataFrame with 6 rows and 3 columns

	gtissue	ptissue	count
	<Rle>	<character>	<integer>
1	bladder	bladder	2055
2	bladder	kidney	1496
3	bladder	NA	997
4	kidney	bladder	97
5	kidney	kidney	192
6	kidney	NA	279

If we want more information per gene, e.g. suppose we want to compute the average signal per gene of peaks nearby, we need to group twice, once also by gene ID, and the second time integrating over gene ID. While here we add a few more lines of code, performing such an operation with base Bioconductor functions would require adding code to perform the loops, adding many intermediate variables to store results, etc.

```
tss %>%
  join_overlap_left(pks, maxgap=500) %>%
  group_by(gtissue, ptissue, gene_id) %>% # need per gene stats
  summarize(num_overlaps = n(), signal = mean(signal)) %>%
  as_tibble() %>% # DataFrame to tibble for further processing
  group_by(gtissue, ptissue) %>%
  summarize(sum_any_overlaps = sum(num_overlaps > 0),
            mean_signal=mean(signal))
```

```
`summarise()` has regrouped the output.
i Summaries were computed grouped by gtissue and ptissue.
i Output is grouped by gtissue.
i Use `summarise(.groups = "drop_last")` to silence this message.
i Use `summarise(.by = c(gtissue, ptissue))` for per-operation grouping
  (`?dplyr::dplyr_by`) instead.
```

```
# A tibble: 6 x 4
# Groups:   gtissue [2]
  gtissue ptissue sum_any_overlaps mean_signal
<chr>    <chr>          <int>         <dbl>
1 bladder bladder      2055          1.18
2 bladder kidney      1496          0.936
3 bladder <NA>         997           NA
4 kidney  bladder       97           0.948
5 kidney  kidney       192          0.733
6 kidney  <NA>        279           NA
```

What's wrong with this analysis?

- We didn't figure out the expressed promoter, we just looked at the left or rightmost isoform (for + or - strand genes, respectively).

9 RNA-seq EDA

Objective: learn how to make basic EDA plots (plotting scaled counts) of RNA-seq data using tidy-style verbs.

The *tidybulk* Bioconductor package and associated [tidy transcriptomics](#) ecosystem (`tidySummarizedExperiment`, `tidySingleCell`, `tidyseurat`, etc.) provide a bridge between the *SummarizedExperiment*-style representation of matrix data with attached metadata, to the tidy-style representation of data (one row per observation). For more details on the package, check out the [tidybulk](#) website, and the publication: Mangiola et al. (2021).

Here we will briefly examine the difference between tidy manipulation of these matrix-style objects compared to how these objects would be manipulated in other Bioconductor workflows (e.g. DESeq2, edgeR, or limma workflows). As you will see, there are multiple ways to generate the same or similar exploratory plots and analyses. we recommend to consider which is more appropriate to your purpose. For package code, you may prefer to have fewer dependencies, going with core Bioconductor objects and packages. For scripting, the “fluent” and piped style of *tidybulk* may be preferable, and easier for others to read and modify your code at a future date.

Here we just compare code for some basic tasks, scaling counts and performing DE with DESeq2. However, note that *tidybulk* has many functionalities implemented, including dimension reduction and visualization (PCA, MDS, tSNE, UMAP), clustering, gene set testing, cell type composition analysis and cell abundance testing, unwanted variation modeling, imputation, etc.

Start by loading this RNA-seq dataset from the following paper: King and Klose (2017) “The pioneer factor OCT4 requires the chromatin remodeller BRG1 to support gene regulatory element function in mouse embryonic stem cells” <https://doi.org/10.7554/eLife.22631>. The experiment focused on OCT4 as it is a “core pluripotency transcription factor” which “occupies sites that would otherwise be inaccessible and is required to shape the occupancy of additional pluripotency transcription factors”.

In this experiment, transcription in mouse embryonic stem cells (ESC) was compared with and without OCT4. This was done using a conditional mouse ESC line where treatment with a compound leads to loss of OCT4 expression. The experiment also involved the same approach to another transcription factor BRG1, but we focus here on the OCT4 samples.

First we load the metadata about the samples:

```
library(oct4)
dir <- system.file("extdata", package="oct4")
coldata <- read.csv(file.path(dir,"coldata.csv"))
coldata
```

	names	line	condition
1	SRX2236945	OCT4	untrt
2	SRX2236946	OCT4	untrt
3	SRX2236947	OCT4	untrt
4	SRX2236948	OCT4	trt
5	SRX2236949	OCT4	trt
6	SRX2236950	OCT4	trt
7	SRX2236951	BRG1	untrt
8	SRX2236952	BRG1	untrt
9	SRX2236953	BRG1	untrt
10	SRX2236954	BRG1	trt
11	SRX2236955	BRG1	trt
12	SRX2236956	BRG1	trt

```
coldata$files <- file.path(dir, "quants", coldata$names, "quant.sf.gz")
```

Read in the count data with *tximeta*, which automatically imports the information about the gene provenance (which transcripts were used to quantify the gene and isoform abundance). We then summarize the quantification to the gene-level, and add the gene SYMBOL.

```
library(tximeta)
se <- tximeta(coldata)
gse <- summarizeToGene(se)
library(org.Mm.eg.db)
gse <- addIds(gse, "SYMBOL")
```

For this workflow, we don't need the inferential replicates (about uncertainty regarding the quantification), so we keep just the counts, abundances (TPM), and gene lengths. We also manipulate the metadata a little bit.

```
library(SummarizedExperiment)
assayNames(gse)
assays(gse) <- assays(gse)[1:3]
gse$rep <- rep(1:3, 4)
colnames(gse) <- paste(gse$line,gse$condition,gse$rep,sep="-")
```

```
assay(gse, "counts") <- round(assay(gse, "counts")) # for DE consistency
# save for easy loading later
save(gse, file="data/oct4_obj.rda")
```

```
library(SummarizedExperiment)
load("data/oct4_obj.rda")
```

The dataset looks like this (remember it has untreated and treated samples for both OCT4 and BRG1).

```
gse
```

```
class: RangedSummarizedExperiment
dim: 53697 12
metadata(7): tximetaInfo quantInfo ... txdbInfo assignRanges
assays(3): counts abundance length
rownames(53697): ENSMUSG000000000001.4 ENSMUSG000000000003.15 ...
               ENSMUSG00000117654.1 ENSMUSG00000117655.1
rowData names(3): gene_id tx_ids SYMBOL
colnames(12): OCT4-untrt-1 OCT4-untrt-2 ... BRG1-trt-2 BRG1-trt-3
colData names(4): names line condition rep
```

We will be interested in the gene set from the Gene Ontology project, which describes maintenance of pluripotency. We can extra this from the mouse organism data package with the following three lines of code:

```
library(AnnotationDbi)
library(org.Mm.eg.db)
```

```
# pluripotency
tab <- AnnotationDbi::select(org.Mm.eg.db, "GO:0019827", "SYMBOL", "GO")
```

'select()' returned 1:many mapping between keys and columns

```
tab <- tab[!duplicated(tab$SYMBOL),]
pluri <- tab$SYMBOL
```

Now we start with *tidybulk* code, comparing in turn to base R. Loading *tidySummarizedExperiment* allows us to operate on the `gse` object using familiar “tidy” verbs, thanks to an abstraction described in the [tidySummarizedExperiment](#) documentation. We filter the samples that correspond to the OCT4 experiment, and then modify the sample names. Samples are referred to with the special `.sample` string, while features are referred to with `.feature`, so here we create a new variable to plot the samples `sample_name`.

```
library(tidySummarizedExperiment)
library(dplyr)
library(stringr)
oct4 <- gse %>%
  filter(line == "OCT4") %>%
  mutate(sample_name = select(., .sample) %>% # this is a hack, .sample seems broken with mu
    str_remove("OCT4-") %>%
    factor(levels = unique(.)),
    condition = condition %>% factor(c("untrt", "trt")))
```

```
Warning: `when()` was deprecated in purrr 1.0.0.
i Please use `if` instead.
i The deprecated feature was likely used in the tidySummarizedExperiment
  package.
  Please report the issue at
  <https://github.com/stemangiola/tidySummarizedExperiment/issues>.
```

```
Warning: There was 1 warning in `dplyr::mutate()`.
i In argument: `sample_name = select(., .sample) %>% str_remove("OCT4-") %>%
  factor(levels = unique(.))`.
Caused by warning in `stri_replace_first_regex()`:
! argument is not an atomic vector; coercing
```

```
Warning: There was 1 warning in `dplyr_fn()`.
i In argument: `sample_name = select(., .sample) %>% str_remove("OCT4-") %>%
  factor(levels = unique(.))`.
Caused by warning in `stri_replace_first_regex()`:
! argument is not an atomic vector; coercing
```

Note, the third line in the `mutate` call uses the `unique` function to set the levels: the consequence of this is that the `sample_name` factor will have levels in the order in which they are present in the current character vector and not alphabetical.

```
oct4
```

```
class: RangedSummarizedExperiment
dim: 53697 6
metadata(9): txmetaInfo quantInfo ... latest_filter_scope_report
  latest_mutate_scope_report
assays(3): counts abundance length
rownames(53697): ENSMUSG000000000001.4 ENSMUSG000000000003.15 ...
  ENSMUSG000000117654.1 ENSMUSG000000117655.1
rowData names(3): gene_id tx_ids SYMBOL
colnames(6): OCT4-untrt-1 OCT4-untrt-2 ... OCT4-trt-2 OCT4-trt-3
colData names(5): names line condition rep sample_name
```

tidybulk provides access to many steps in bulk analysis, including filtering and count scaling. For details on what is happening behind the scene, see the help, e.g. `?keep_abundant` describes that it makes use of `edgeR::filterByExpr`.

```
library(tidybulk)
oct4 <- oct4 %>%
  keep_abundant(factor_of_interest = condition) %>%
  scale_abundance(method="RLE") # DESeq2 scaling
```

Warning: The ``factor_of_interest`` argument of ``keep_abundant()`` is deprecated as of tidybulk 2.0.0.

i Please use the ``formula_design`` argument instead.

i The argument `'factor_of_interest'` is deprecated and will be removed in a future release. Please use the `'design'` or `'formula_design'` argument instead.

Warning: The ``factor_of_interest`` argument of ``identify_abundant()`` is deprecated as of tidybulk 2.0.0.

i Please use the ``formula_design`` argument instead.

i The argument `'factor_of_interest'` is deprecated and will be removed in a future release. Please use the `'design'` or `'formula_design'` argument instead.

i The deprecated feature was likely used in the tidybulk package.

Please report the issue at <https://github.com/stemangiola/tidybulk/issues>.

It is straightforward to pipe the data directly into plots:

[illegible]

```
gene_idx <- oct4 %>% pivot_transcript() %>% pull(.feature)
head(gene_idx)
```

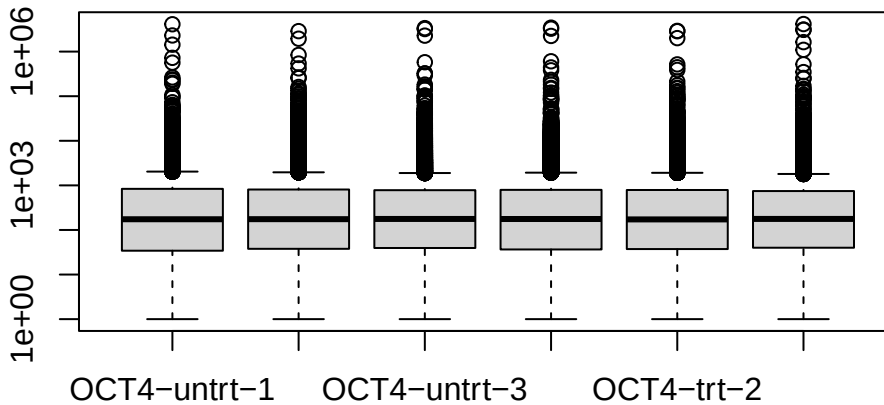
The equivalent code in DESeq2. Understanding this code requires knowledge that `boxplot` plots columns of a matrix.

64

using counts and average transcript lengths from tximeta

using 'avgTxLength' from assays(dds), correcting for library size

```
boxplot(counts(dds, normalized=TRUE) + 1, log="y")
```



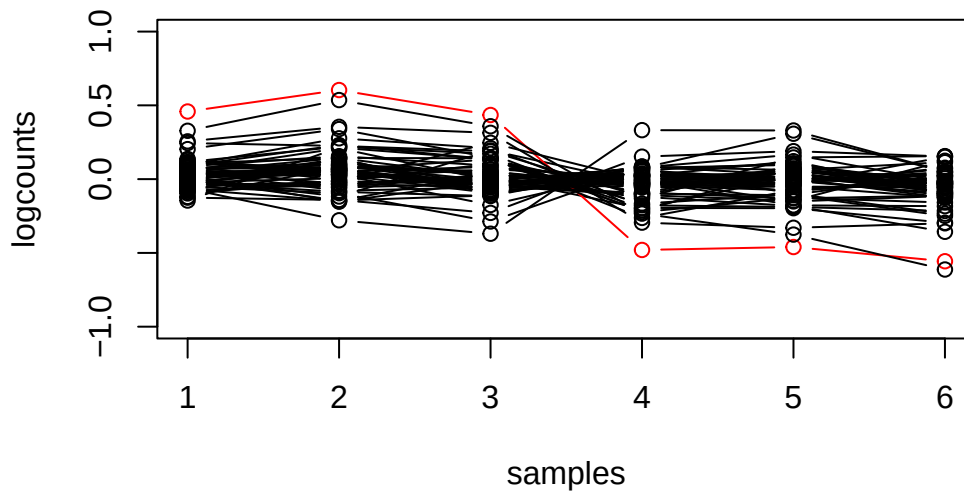
We can also make more interesting plots. E.g. for the genes involved in pluripotency, make a line plot, highlighting OCT4. In addition, center the log counts for each gene (subtract the mean of log counts across samples).

```
oct4 %>%  
  filter(SYMBOL %in% pluri) %>%  
  mutate(logcounts = log10(counts_scaled + 1)) %>%  
  mutate(Oct4 = ifelse(SYMBOL == "Pou5f1", "red", "black")) %>%  
  group_by(.feature) %>%  
  mutate(logcounts = logcounts - mean(logcounts)) %>%  
  ungroup() %>%  
  ggplot(aes(sample_name, logcounts, group=.feature, color=Oct4)) +  
  geom_point() +  
  geom_line() +  
  scale_color_identity()
```

tidySummarizedExperiment says: A data frame is returned for independent data analysis.

The equivalent code for base R requires defining more intermediate variables and control flow code (the `for` loop). While there's nothing particularly right or wrong about the two choices, the above prioritizes the operations in a way that is human readable. In some cases, e.g. performing linear algebra operations on matrices, base R code may prove to be more efficient, which is a consideration for what to use in package source code.

66



We can test for differential expression with DESeq2:

```
res <- dds[gene_idx,] %>%
  DESeq(quiet=TRUE) %>%
  results()
```

Or equivalently with *tidybulk*:

```
oct4 <- oct4 %>%
  test_differential_abundance(~condition, method="deseq2")
```

=====
 tidybulk says: All testing methods use raw counts, irrespective of if scale_abundance or adjust_abundance have been calculated. Therefore, it is essential to add covariates such as batch effects (if applicable) in the formula.
 =====

converting counts to integer mode

estimating size factors

estimating dispersions

gene-wise dispersion estimates

mean-dispersion relationship

final dispersion estimates

fitting model and testing

tidybulk says: to access the DE object do ``metadata(.)$tidybulk$deseq2_object``

tidybulk says: to access the raw results (fitted GLM) do ``metadata(.)$tidybulk$deseq2_fit``

This message is displayed once per session.

```
tidy_res <- oct4 %>%  
  pivot_transcript()
```

Because we have filtered the two objects identically, we obtain the same test results:

```
all.equal(rownames(res), tidy_res$.feature)
```

```
[1] TRUE
```

```
table(base_sig = res$padj < .1, tidy_sig = tidy_res$padj < .1)
```

	tidy_sig	
base_sig	FALSE	TRUE
FALSE	17325	640
TRUE	418	3094

Finally, we build up to a more interesting plot. Suppose we now want to split the genes involved in pluripotency by the DE result (the significance and LFC), and then add the gene symbol to the side.

We begin by building the dataset:

```
plot_data <- oct4 %>%  
  filter(SYMBOL %in% pluri) %>%  
  mutate(logcounts = log10(counts_scaled + 1)) %>%  
  mutate(Oct4 = ifelse(SYMBOL == "Pou5f1", "red", "black")) %>%  
  group_by(.feature) %>%  
  mutate(logcounts = logcounts - mean(logcounts)) %>%  
  ungroup() %>%  
  mutate(gene_type = case_when(  
    padj < .1 & log2FoldChange > 0 ~ "up",  
    padj < .1 & log2FoldChange < 0 ~ "down",  
    TRUE ~ "null"))
```

`tidySummarizedExperiment` says: A data frame is returned for independent data analysis.

Now we repeat the code from before, but now faceting by `gene_type`. Furthermore, we use `geom_text_repel` to add labels to the right side.

```
library(ggrepel)
plot_data %>%
  filter(gene_type != "null") %>%
  ggplot(aes(sample_name, logcounts, group=.feature, color=Oct4)) +
  geom_point() +
  geom_line() +
  geom_text_repel(data=plot_data %>%
    filter(sample_name == "trt-3", gene_type != "null"),
    aes(sample_name, logcounts, label=SYMBOL),
    nudge_x=.5, seed=1, max.overlaps=Inf) +
  scale_color_identity() +
  facet_wrap(~gene_type) +
  scale_x_discrete(expand = expansion(add = 2)) +
  xlab("sample")
```

[illegible]

10 SNP position in peaks

Objective: determine the position of a set of SNPs in peaks, that is, determine the relative positions of those SNPs that overlap the peaks.

We again will use the ENCODE kidney H3K27ac ChIP-seq peaks used in the previous analysis ([Dunham et al. 2012](#)). We will create some artificial SNPs: this analysis could generalize to any time that we have two ranges, where we are interested in the relative position of one set of ranges (SNPs) within the other set of ranges (peaks).

```
library(AnnotationHub)
ah <- AnnotationHub()
kidney_pks <- ah[["AH43443"]]
```

We will filter the peaks to standard chromosomes, and include the same cutoffs we used in the previous analysis.

```
suppressPackageStartupMessages(library(GenomeInfoDb))
pks <- kidney_pks
pks <- keepStandardChromosomes(pks)
```

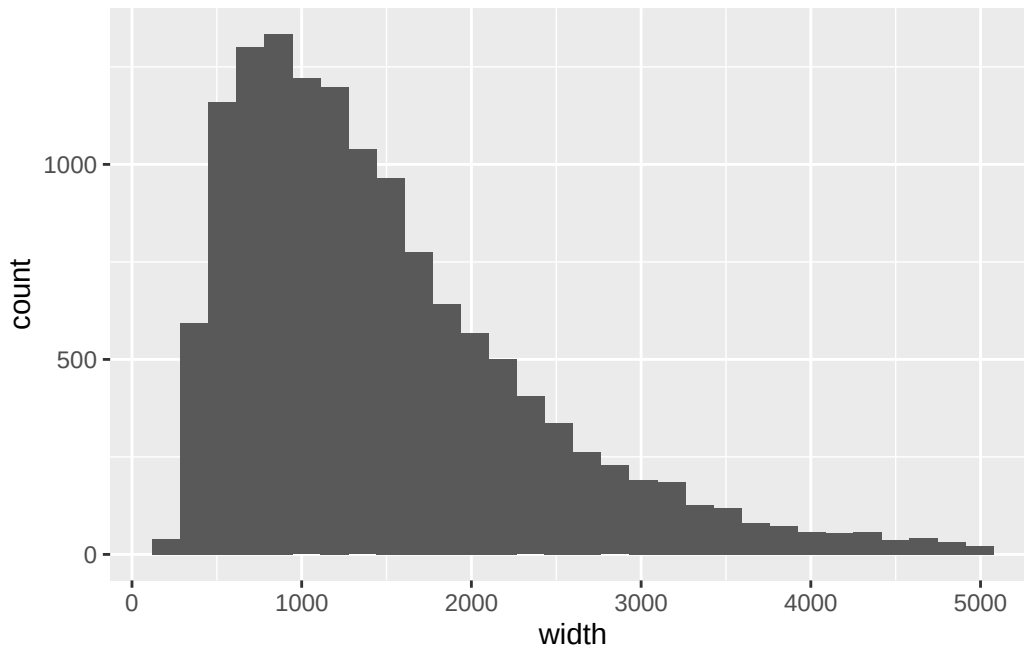
Loading required namespace: GenomicRanges

```
suppressPackageStartupMessages({
  library(dplyr)
  library(tibble)
  library(plyranges)
})
q_thr <- 3
s_thr <- 9
pks <- pks %>%
  filter(qValue > q_thr & signalValue > s_thr) %>%
  sort()
```

A histogram of peak width:

```
library(ggplot2)
pks %>% as_tibble %>%
  filter(width < 5000) %>%
  ggplot(aes(width)) +
  geom_histogram()
```

`stat\_bin()` using `bins = 30`. Pick better value `binwidth`.



Let's subset to peaks that are less than 2000 in width:

```
pks <- pks %>% filter(width < 2000)
```

Now, we may be interested in SNPs that are associated with regulatory function. While we could find these from a database of chromatin QTL or functionally validated variants, here we will just create a simulated set of SNPs for demonstration.

```
set.seed(1)
snps <- pks %>%
  slice(sample.int(n(), 2000)) %>%
  anchor_5p() %>%
  mutate(start=start + floor(runif(n(),0,width))) %>%
  mutate(width=1)
snps
```

GRanges object with 2000 ranges and 6 metadata columns:

	seqnames	ranges	strand	name	score	signalValue
	<Rle>	<IRanges>	<Rle>	<character>	<numeric>	<numeric>
[1]	chr1	222886667	*	Rank_786	943	28.06788
[2]	chr17	1251302	*	Rank_11154	238	10.45065
[3]	chr9	75179753	*	Rank_8060	305	12.52016
[4]	chrX	70401571	*	Rank_9602	266	11.38197
[5]	chr20	37615472	*	Rank_14922	190	9.52741
...	...	...	...	...	...	...
[1996]	chr9	14316343	*	Rank_15136	188	9.11715
[1997]	chr18	74800298	*	Rank_15685	181	9.02626
[1998]	chr19	52641902	*	Rank_2464	606	19.64891
[1999]	chr2	128642862	*	Rank_15514	184	9.18719
[2000]	chr6	151005236	*	Rank_13672	203	9.07738

	pValue	qValue	peak
	<numeric>	<numeric>	<numeric>
[1]	94.3437	89.9679	267
[2]	23.8843	21.0424	458
[3]	30.5751	27.5376	616
[4]	26.6991	23.7679	455
[5]	19.0203	16.3297	414
...	...	...	...
[1996]	18.8452	16.1647	461
[1997]	18.1808	15.5192	173
[1998]	60.6790	56.9445	1281
[1999]	18.4512	15.7837	125
[2000]	20.3511	17.6170	887

seqinfo: 25 sequences (1 circular) from hg19 genome

These should all overlap peaks:

```
snps %>%
  mutate(n_overlaps = count_overlaps(., pks)) %>%
  summarize(tab=table(n_overlaps))
```

DataFrame with 1 row and 2 columns

	tab.n_overlaps	tab.Freq
	<factor>	<integer>
1	1	2000

We now have a bit of a hack for this analysis: we add the start position and width of the peak as additional columns of metadata. This is because otherwise, we will lose the start position when we perform the overlap of SNPs in peaks, and we need it for later.

```
pks_trim <- pks %>%  
  select(name) %>%  
  mutate(peak_start=start, peak_width=width)
```

We can then obtain overlaps of SNPs with peaks, and add the relative position of the SNP within the peak extent. We use 1-based indexing, such that if the SNP is the same as the leftmost basepair of the peak, it gets counted as position 1.

```
o <- snps %>%  
  join_overlap_inner(pks_trim) %>%  
  mutate(rel_pos = start - peak_start + 1,  
         rel_frac = (rel_pos - 1) / (peak_width - 1))
```

Check our expectations about these new columns:

```
all(o$rel_pos >= 1)
```

```
[1] TRUE
```

```
all(o$rel_frac >= 0)
```

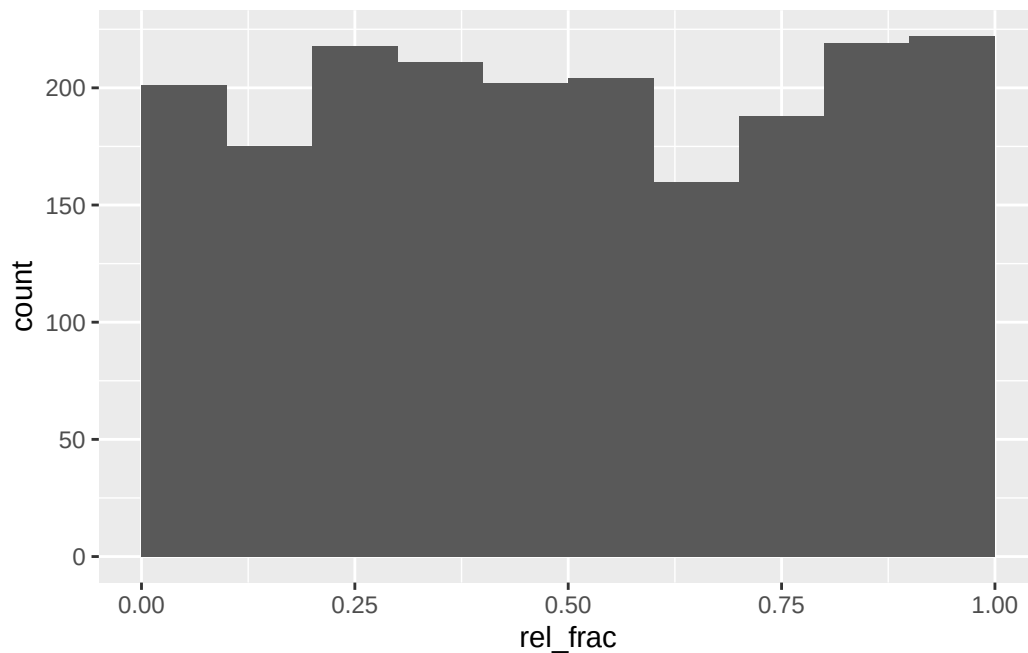
```
[1] TRUE
```

```
all(o$rel_frac <= 1)
```

```
[1] TRUE
```

Finally, we can compute a histogram of where the SNPs fall in the peaks:

```
o %>%  
  as_tibble() %>%  
  ggplot(aes(rel_frac)) +  
  geom_histogram(breaks=0:10/10)
```



Now stratifying by width of peak:

```
quantile(o$peak_width, 0:3/3)
```

```

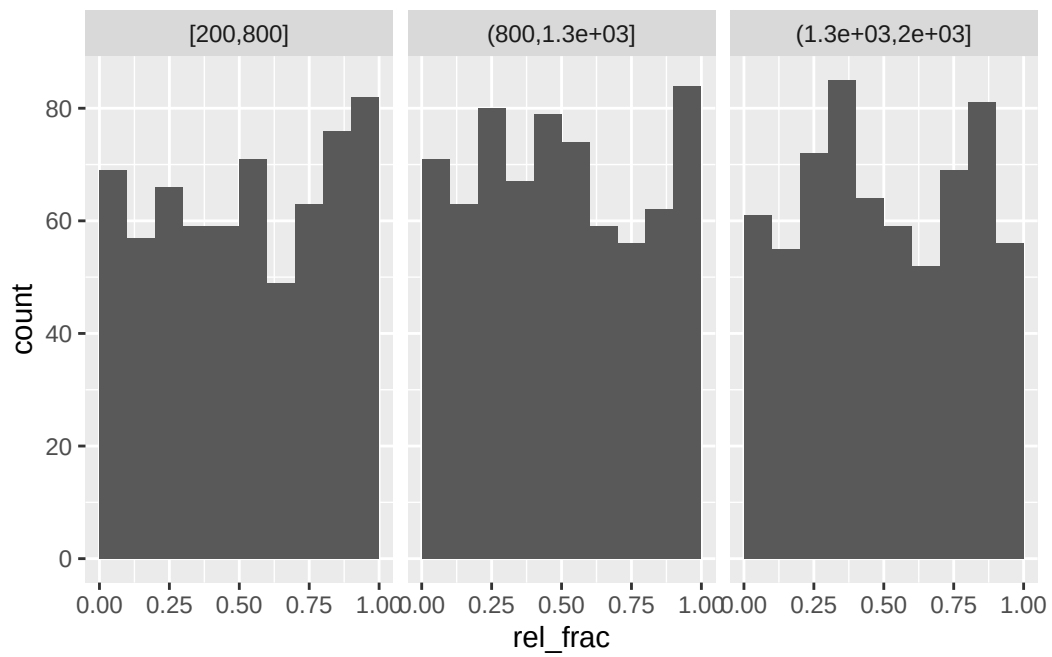
0% 33.33333% 66.66667% 100%
246.0000 813.3333 1289.6667 1998.0000

```

```

o %>%
  mutate(width_bin = cut(peak_width,
                        breaks=c(200,800,1300,2000),
                        include.lowest=TRUE)) %>%
  as_tibble() %>%
  ggplot(aes(rel_frac)) +
  geom_histogram(breaks=0:10/10) +
  facet_wrap(~width_bin)

```



11 Gene ranges in tximeta

Objective: explore how genes are located in the genome during summarization with tximeta.

The tximeta package (Love et al. 2020) imports gene expression quantification into Bioconductor. The package is built around a few main ideas:

1. transcript-level quantification improves accuracy even for gene-level inference (Soneson, Love, and Robinson 2015)
2. summarization to gene level is a common use case
3. reference sequence hashes (think of a “barcode” that uniquely identifies the cDNA sequences) can be used to identify the provenance and release information of annotation

tximeta uses reference sequence hashes embedded in metadata files provided with Salmon (Patro et al. 2017) quantification output to automatically attach genomic range information, transcript-to-gene mapping, and genome build information to SummarizedExperiments. This is done automatically, on behalf of the user, without needing to specify the provenance or release numbers used during quantification. tximeta can also be used in combination with de novo transcriptomes or less common transcriptomes (see Love et al. (2020) or package vignette).

When data is summarized to the gene level, tximeta by default uses the `genes()` function to locate the genes in the genome. This provides a genomic range which starts with the leftmost position of any isoform and ends with the rightmost position of any isoform. This however may not be a good representation of the locus of transcription.

In this chapter, we explore an alternative way to assign ranges in tximeta’s summarization step, which is based on isoform abundance. We use `plyranges` and `plotgardener` packages to explore the differences between these two options for range assignment.

Data description: the *macrophage* package provides Salmon quantification files for a set of 24 RNA-seq samples from Alasoo et al. (2018). Data for six female human donors is available in the package, with gene expression at baseline, after IFN-gamma stimulation, after exposure to *Salmonella*, and after a combination of the two treatments. We won’t focus on the design of the experiment itself, but we will use transcript-level quantification data to explore the differences between the two range assignment options.

The following code chunk creates a `coldata` table describing the samples.

```

library(macrophage)
library(dplyr)
library(tximeta)
dir <- system.file("extdata", package="macrophage")
coldata <- read.csv(file.path(dir, "coldata.csv")) %>%
  mutate(files = file.path(dir, "quants", names, "quant.sf.gz"),
         condition = factor(condition_name),
         condition = relevel(condition, "naive")) %>%
  select(files, names=sample_id, condition, line_id) %>%
  slice(1:4) # subset to a few samples
coldata %>%
  dplyr::select(-files)

```

```

  names    condition line_id
1 diku_A      naive  diku_1
2 diku_B      IFNg   diku_1
3 diku_C      SL1344 diku_1
4 diku_D IFNg_SL1344 diku_1

```

Importing the transcript-level data as a SummarizedExperiment:

```
se <- tximeta(coldata, dropInfReps=TRUE)
```

We can then see the ranges of the transcripts and the genome build information:

```

library(SummarizedExperiment)
rowRanges(se)

```

GRanges object with 205870 ranges and 3 metadata columns:

	seqnames	ranges	strand	tx_id	gene_id
	<Rle>	<IRanges>	<Rle>	<integer>	<CharacterList>
ENST00000456328.2	chr1	11869-14409	+	1	ENSG00000223972.5
ENST00000450305.2	chr1	12010-13670	+	2	ENSG00000223972.5
ENST00000488147.1	chr1	14404-29570	-	9483	ENSG00000227232.5
ENST00000619216.1	chr1	17369-17436	-	9484	ENSG00000278267.1
ENST00000473358.1	chr1	29554-31097	+	3	ENSG00000243485.5
...	...	...	...	...	...
ENST00000361681.2	chrM	14149-14673	-	206692	ENSG00000198695.2
ENST00000387459.1	chrM	14674-14742	-	206693	ENSG00000210194.1
ENST00000361789.2	chrM	14747-15887	+	206684	ENSG00000198727.2

```

ENST00000387460.2      chrM 15888-15953      + |      206685 ENSG00000210195.2
ENST00000387461.2      chrM 15956-16023      - |      206694 ENSG00000210196.2
      tx_name
      <character>
ENST00000456328.2 ENST00000456328.2
ENST00000450305.2 ENST00000450305.2
ENST00000488147.1 ENST00000488147.1
ENST00000619216.1 ENST00000619216.1
ENST00000473358.1 ENST00000473358.1
      ...      ...
ENST00000361681.2 ENST00000361681.2
ENST00000387459.1 ENST00000387459.1
ENST00000361789.2 ENST00000361789.2
ENST00000387460.2 ENST00000387460.2
ENST00000387461.2 ENST00000387461.2
-----
seqinfo: 25 sequences (1 circular) from hg38 genome

```

```
seqinfo(se)
```

Seqinfo object with 25 sequences (1 circular) from hg38 genome:

```

seqnames seqlengths isCircular genome
chr1      248956422      FALSE   hg38
chr2      242193529      FALSE   hg38
chr3      198295559      FALSE   hg38
chr4      190214555      FALSE   hg38
chr5      181538259      FALSE   hg38
...      ...      ...      ...
chr21     46709983      FALSE   hg38
chr22     50818468      FALSE   hg38
chrX      156040895      FALSE   hg38
chrY      57227415      FALSE   hg38
chrM      16569         TRUE    hg38

```

The default approach to assign ranges during summarization (`rowRanges` will represent the total extent of all isoforms of the gene):

```
gse_default <- summarizeToGene(se)
```

loading existing TxDb created: 2026-10-08 14:37:47

obtaining transcript-to-gene mapping from database

loading existing gene ranges created: 2026-10-08 14:37:50

assignRanges='range': gene ranges assigned by total range of isoforms
see details at: [?summarizeToGene,SummarizedExperiment-method](#)

summarizing abundance

summarizing counts

summarizing length

And an alternative method to assign ranges by isoform abundance (`rowRanges` will represent the extent of the most abundant isoform):

```
gse <- summarizeToGene(se, assignRanges="abundant")
```

loading existing TxDb created: 2026-10-08 14:37:47

obtaining transcript-to-gene mapping from database

loading existing gene ranges created: 2026-10-08 14:37:50

assignRanges='abundant': gene ranges assigned by isoform abundance
see details at: [?summarizeToGene,SummarizedExperiment-method](#)

summarizing abundance

summarizing counts

summarizing length

Note that these two steps provide identical counts, abundance, etc. They only differ by their `rowRanges`.

```
all.equal(assay(gse_default), assay(gse))
```

```
[1] TRUE
```

We can now compare the 5' location of the ranges using the two approaches.

```
library(plyranges)
# plyranges to locate the 5' start using default approach
default_5p <- gse_default %>%
  rowRanges() %>%
  anchor_5p() %>%
  mutate(width=1) %>%
  start()
```

Pull out information from the SummarizedExperiment: the average abundance over the samples, and the absolute distance between the 5' start of the new and the old approach.

```
gene_dat <- gse %>%
  rowRanges() %>%
  anchor_5p() %>%
  mutate(width=1) %>%
  mutate(
    ave_tpm = rowMeans(assay(gse, "abundance")),
    dist = abs(start - default_5p))
# how many genes have same 5' start?
table(gene_dat$dist == 0)
```

```
FALSE TRUE
11775 46519
```

While most genes have the same 5' start, a good number have changed their start position.

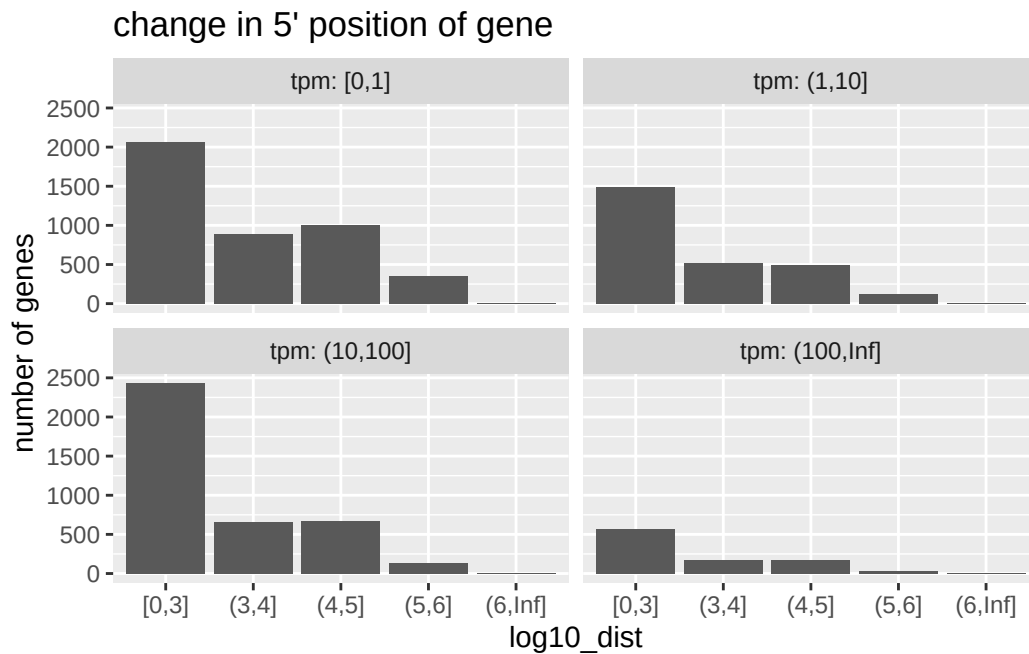
We can summarize the extent of changes by gene expression:

```
library(ggplot2)
gene_dat %>%
  as_tibble() %>%
  filter(dist > 0) %>%
  mutate(
```

```

log10_dist = cut(
  log10(dist),
  breaks=c(0,3:6,Inf),
  include.lowest=TRUE),
tpm = cut(
  ave_tpm,
  breaks=c(0,1,10,100,Inf),
  include.lowest=TRUE)
) %>%
group_by(log10_dist, tpm) %>%
tally() %>%
ggplot(aes(log10_dist, n)) +
geom_col() +
ggtitle("change in 5' position of gene") +
ylab("number of genes") +
facet_wrap(~tpm, labeller=label_both)

```



Considerations:

- The isoform-abundance based representation of the gene is likely a much more accurate description of the locus of transcription than the total gene extent.
- The isoform-abundance 5' start can be very far away from the total gene extent 5' start.

- Gene-level summary is an incomplete representation of transcription, and perhaps it may be better to perform analysis on the transcript level, e.g. DTE (see `swish` in fishpond package or `catchSalmon/catchKallisto` in edgeR) or DTU (see `DEXSeq`, `satuRn`, `diffSplice` in limma package, Segers et al. (2023), or the rnaseqDTU workflow).

We conclude this chapter by diagramming the difference between the two approaches for a single gene, using plotgardener (Kramer et al. 2022).

We begin by building a custom genome assembly for plotgardener functions. Note the use of `tximeta::retrieveDb` to pull the exact TxDb used in quantification of the data for representation of the ranges in the plots.

```
library(plotgardener)
library(org.Hs.eg.db)
# retrieves the correct TxDb/EnsDb for use in plots, etc.
txdb <- tximeta::retrieveDb(gse)
new_assembly <- assembly(
  Genome = "hg38",
  TxDb = txdb,
  OrgDb = org.Hs.eg.db,
  gene.id.column = "GENEID",
  display.column = "GENEID",
  BSgenome = NULL
)
```

We pick a random gene with a large distance and high expression:

```
set.seed(5)
gene <- gene_dat %>%
  filter(strand == "-", dist > 1e5, ave_tpm > 100) %>%
  dplyr::slice(sample(n(),1))
```

The following sets up the parameters for the following plotgardener command.

```
par <- pgParams(
  chrom = seqnames(gene) %>% as.character(),
  chromstart = round((start(gene) - 7e5) / 1e5) * 1e5,
  chromend = round((end(gene) + 7e5) / 1e5) * 1e5,
  assembly = new_assembly,
  just = c("left", "bottom")
)
```

Examine the isoform proportions for the top 5 isoforms of this gene:

```
props <- gene$iso_prop[[1]] %>%
  sort(decreasing=TRUE) %>%
  head(5)
props
```

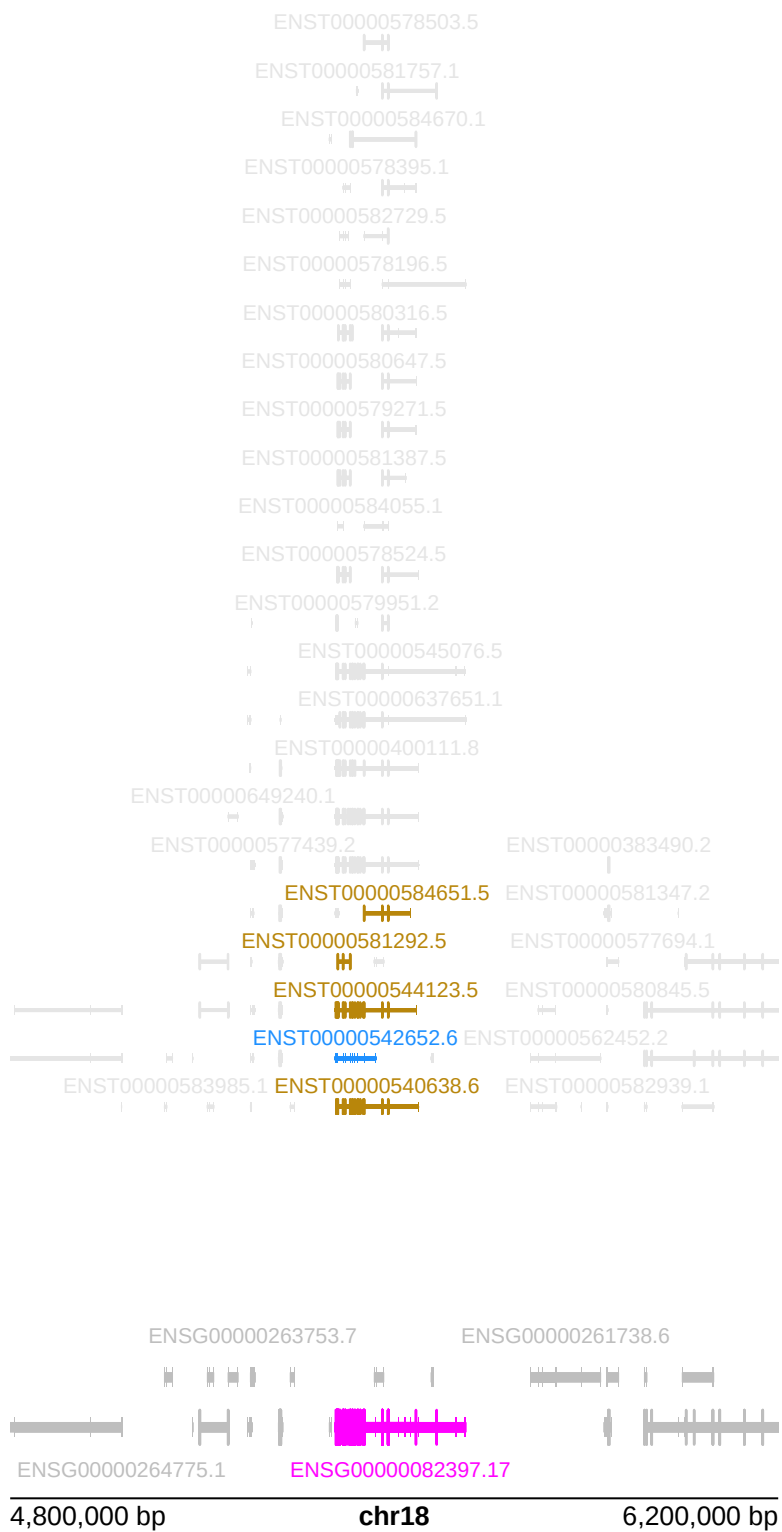
```
ENST00000542652.6 ENST00000540638.6 ENST00000544123.5 ENST00000584651.5
      0.35557166      0.31174119      0.07755495      0.06530776
ENST00000581292.5
      0.03857342
```

Highlight the gene, and the top 5 isoforms in the following plot.

```
gene_hilite <- data.frame(
  gene=gene$gene_id,
  color="magenta"
)
txp_hilite <- data.frame(
  transcript=names(props),
  color=rep(c("dodgerblue","darkgoldenrod"),c(1,length(props)-1))
)
```

Building the plotgardener plot:

```
pageCreate(width = 5, height = 10, showGuides = FALSE, default.units = "inches")
plotGenes(
  params = par, x = 0.5, y = 9, width = 4, height = 1,
  geneHighlights = gene_hilite
)
plotTranscripts(
  params = par, x = 0.5, y = 7, width = 4, height = 10,
  transcriptHighlights = txp_hilite, fill="grey90"
)
plotGenomeLabel(
  params = par, x = 0.5, y = 9, length = 4,
  just = c("left", "top")
)
```



References

- Alasoo, Kaur, Julia Rodrigues, Subhankar Mukhopadhyay, Andrew J. Knights, Alice L. Mann, Kousik Kundu, Christine Hale, Gordon Dougan, Daniel J. Gaffney, and H. I. P. S. C. I. Consortium. 2018. “Shared Genetic Effects on Chromatin and Gene Expression Indicate a Role for Enhancer Priming in Immune Response.” *Nature Genetics* 50 (3): 424–31. <https://doi.org/10.1038/s41588-018-0046-7>.
- Bickel, Peter J., Nathan Boley, James B. Brown, Haiyan Huang, and Nancy R. Zhang. 2010. “Subsampling Methods for Genomic Inference.” *The Annals of Applied Statistics* 4 (4): 1660–97. <https://doi.org/10.1214/10-%7BAOAS363%7D>.
- Davis, Eric S, Wancen Mu, Stuart Lee, Mikhail G Dozmorov, Michael I Love, and Douglas H Phanstiel. 2023. “matchRanges: generating null hypothesis genomic ranges via covariate-matched sampling.” *Bioinformatics* 39 (5). <https://doi.org/10.1093/bioinformatics/btad197>.
- Dunham, I. et al. 2012. “An integrated encyclopedia of DNA elements in the human genome.” *Nature* 489 (7414): 57–74.
- Friedman, Jerome H., Trevor Hastie, and Rob Tibshirani. 2010. “Regularization Paths for Generalized Linear Models via Coordinate Descent.” *Journal of Statistical Software* 33 (1): 1–22. <https://doi.org/10.18637/jss.v033.i01>.
- King, Hamish W, and Robert J Klose. 2017. “The Pioneer Factor OCT4 Requires the Chromatin Remodeller BRG1 to Support Gene Regulatory Element Function in Mouse Embryonic Stem Cells.” Edited by Irwin Davidson. *eLife* 6 (March): e22631. <https://doi.org/10.7554/eLife.22631>.
- Kramer, Nicole E, Eric S Davis, Craig D Wenger, Erika M Deoudes, Sarah M Parker, Michael I Love, and Douglas H Phanstiel. 2022. “Plotgardener: cultivating precise multi-panel figures in R.” *Bioinformatics* 38 (7): 2042–45. <https://doi.org/10.1093/bioinformatics/btac057>.
- Lawrence, Michael, Wolfgang Huber, Hervé Pagès, Patrick Aboyoun, Marc Carlson, Robert Gentleman, Martin T Morgan, and Vincent J Carey. 2013. “Software for Computing and Annotating Genomic Ranges.” *PLoS Comput. Biol.* 9 (8): e1003118.
- Lee, Stuart, Dianne Cook, and Michael Lawrence. 2019. “Plyranges: A Grammar of Genomic Data Transformation.” *Genome Biol.* 20 (1): 4.
- Lee, Stuart, Michael Lawrence, and Michael I Love. 2020. “Fluent Genomics with Plyranges and Tximeta.” *F1000Res.* 9 (February): 109.
- Leong, Hui Sun, Keren Dawson, Chris Wirth, Yaoyong Li, Yvonne Connolly, Duncan L Smith, Caroline R M Wilkinson, and Crispin J Miller. 2014. “A Global Non-Coding RNA System Modulates Fission Yeast Protein Levels in Response to Stress.” *Nat. Commun.* 5 (1): 3947.
- Lonsdale, J. et al. 2013. “The Genotype-Tissue Expression (GTEx) project.” *Nature Genetics* 45 (6): 580–85.

- Love, Michael I., Charlotte Soneson, Peter F. Hickey, Lisa K. Johnson, N. Tessa Pierce, Lori Shepherd, Martin Morgan, and Rob Patro. 2020. "Tximeta: Reference sequence checksums for provenance identification in RNA-seq." *PLOS Computational Biology* 16: e1007664. <https://doi.org/10.1371/journal.pcbi.1007664>.
- Mangiola, Stefano, Ramyar Molania, Ruining Dong, Maria A. Doyle, and Anthony T. Papenfuss. 2021. "Tidybulk: An r Tidy Framework for Modular Transcriptomic Data Analysis." *Genome Biology* 22 (1): 42. <https://doi.org/10.1186/s13059-020-02233-7>.
- Mu, Wancen, Eric S Davis, Stuart Lee, Mikhail G Dozmorov, Douglas H Phanstiel, and Michael I Love. 2023. "bootRanges: flexible generation of null sets of genomic ranges for hypothesis testing." *Bioinformatics* 39 (5). <https://doi.org/10.1093/bioinformatics/btad190>.
- Patro, Rob, Geet Duggal, Michael I. Love, Rafael A. Irizarry, and Carl Kingsford. 2017. "Salmon Provides Fast and Bias-Aware Quantification of Transcript Expression." *Nature Methods*. <https://doi.org/10.1038/nmeth.4197>.
- Rainer, Johannes, Laurent Gatto, and Christian X Weichenberger. 2019. "ensemldb: an R package to create and use Ensembl-based annotation resources." *Bioinformatics* 35 (17): 3151–53. <https://doi.org/10.1093/bioinformatics/btz031>.
- Segers, Alexandre, Jeroen Gilis, Mattias Van Heetvelde, Elfride De Baere, and Lieven Clement. 2023. "Juggling Offsets Unlocks RNA-Seq Tools for Fast Scalable Differential Usage, Aberrant Splicing and Expression Analyses." *bioRxiv*. <https://doi.org/10.1101/2023.06.29.547014>.
- Soneson, Charlotte, Michael I. Love, and Mark Robinson. 2015. "Differential analyses for RNA-seq: transcript-level estimates improve gene-level inferences." *F1000Research* 4. <https://doi.org/10.12688/f1000research.7563.1>.
- Vitting-Seerup, Kristoffer, and Albin Sandelin. 2019. "IsoformSwitchAnalyzeR: Analysis of Changes in Genome-Wide Patterns of Alternative Splicing and Its Functional Consequences." *Bioinformatics* 35 (21): 4469–71.
- Wickham, Hadley, and Garrett Grolemund. 2017. *R for Data Science: Import, Tidy, Transform, Visualize, and Model Data*. 1st ed. Paperback; O'Reilly Media. <http://r4ds.had.co.nz/>.